

**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»**

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

До захисту допущено:

Завідувач кафедри

_____ Олександр РОЛІК

«__» _____ 2026 р.

**Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інформаційне забезпечення
робототехнічних систем»
спеціальності 126 «Інформаційні системи та технології»
на тему: «Інтелектуальна система керування роботизованою рукою із
застосуванням комп'ютерного зору»**

Виконав:

студент IV курсу, групи ІК-23

Павлов Андрій Тарасович

Керівник:

Асистент кафедри ІСТ, PhD

Нікітін Валерій Андрійович

Рецензент:

Старший викладач кафедри ІПІ, PhD

Дифучина Олександра Юріївна

Засвідчую, що у цьому дипломному
проєкті немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____

Київ – 2026 року

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 126 «Інформаційні системи та технології»

Освітньо-професійна програма «Інформаційне забезпечення робототехнічних систем»

ЗАТВЕРДЖУЮ

Завідувач кафедри

_____ Олександр РОЛІК

«___» _____ 2026 р.

ЗАВДАННЯ

на дипломний проєкт студенту

Павлову Андрію Тарасовичку

1. Тема проєкту «Інтелектуальна система керування роботизованою рукою із застосуванням комп'ютерного зору», керівник проєкту Нікітін Валерій Андрійович, асистент кафедри ICT, PhD, затверджені наказом по університету від «___» _____ 2026 р. № ____-с

2. Термін подання студентом проєкту: 8 червня 2026 р.

3. Вихідні дані до проєкту: система безконтактного керування антропоморфним маніпулятором (16 ступенів свободи); обробка відеопотоку від 1280×720 пікселів при 30–35 FPS; час відгуку до 100 мс; розпізнавання 21 точки кисті на відстані 50–80 см; керування сервоприводами SG90/MG90S

через ESP32S3 та драйвер PCA9685 по UART (115200 бод); двоступенева ЕМА-фільтрація для усунення шуму.

4. Зміст пояснювальної записки: анотація, перелік умовних позначень, вступ, опис предметної області, аналіз існуючих рішень та методів керування, проєктування інтелектуальної системи керування, вибір апаратно-технічних та програмних засобів, реалізація інтелектуальної системи керування, тестування інтелектуальної системи керування, загальні висновки, перелік використаних джерел, додатки.

5. Перелік графічного матеріалу: 3D-модель маніпулятора, електрична схема підключення, діаграма послідовності взаємодії компонентів, інтерфейс програми, графіки роботи фільтрів.

6. Дата видачі завдання: 20 квітня 2026 р.

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Аналіз предметної області	20.04.2026	Виконано
2	Аналіз існуючих рішень	24.04.2026	Виконано
3	Формулювання вимог до системи	27.04.2026	Виконано
4	Вибір апаратно-технічної складової	04.05.2026	Виконано
5	Реалізація інформаційної системи	11.05.2026	Виконано
6	Тестування інформаційної системи	15.05.2026	Виконано
7	Оформлення пояснювальної записки	17.05.2026	Виконано

Студент

Андрій ПАВЛОВ

Керівник

Валерій НІКІТІН

АНОТАЦІЯ

Інтелектуальна система керування роботизованою рукою із застосуванням комп'ютерного зору.

Проект містить 71 с. тексту, 11 рисунків, 8 таблиць, посилання на 27 літературних джерел, 1 додаток та 15 конструкторських документів.

Ключові слова: комп'ютерний зір, розпізнавання жестів руки, керування рукою, MediaPipe, OpenCV, ESP32S3, ЕМА-фільтрація, інформаційні системи, інформаційні технології.

Об'єктом роботи є процес взаємодії людини з роботизованим маніпулятором засобами безконтактного жестового керування.

Метою проекту є підвищення інтуїтивності керування, точності та стабільності людино-машинної взаємодії шляхом розробки інтелектуальної системи безконтактного керування антропоморфним роботизованим маніпулятором у режимі реального часу.

У дипломному проекті розроблено програмно-апаратне рішення жестового керування за моделлю «зір – інтерпретація – керування», що включає антропоморфну роботизовану руку та програмне забезпечення для комп'ютерного зору, що забезпечує просторовий трекінг кисті й розрахунок кутів її суглобів. Система забезпечує безперервне копіювання рухів оператора в «режимі тіні» з частотою обробки 30–35 FPS, повністю усуваючи апаратне тремтіння сервоприводів у статиці завдяки алгоритмам двоступеневої ЕМА-фільтрації сигналів.

Практична цінність проекту полягає у створенні функціонального прототипу системи, який може бути використано як основу для розроблення персональних роботів-асистентів, а також для інтеграції у медичні системи та навчально-дослідницькі лабораторні стенди.

SUMMARY

Intelligent Control System for a Robotic Hand Using Computer Vision.

The project contains 71 pages of text, 11 figures, 8 tables, references to 27 literary sources, 1 appendix, and 5 design documents.

Keywords: computer vision, hand gesture recognition, hand control, MediaPipe, OpenCV, ESP32S3, EMA filtering, information systems, information technologies.

The object of the study is the process of human interaction with a robotic manipulator using contactless gesture control methods.

The aim of the project is to improve the intuitiveness, accuracy, and stability of human-machine interaction through the development of an intelligent contactless control system for an anthropomorphic robotic manipulator in real time.

The diploma project presents the development of a hardware–software gesture control system based on the "vision – interpretation – control" model, which includes an anthropomorphic hardware platform and software for detecting and calculating finger joint angles. The system provides continuous mirroring of the operator's movements in "shadow mode" at a processing rate of 30–35 FPS and completely eliminates servo motor jitter in static positions due to a two-stage exponential moving average (EMA) filtering algorithm.

The practical value of the project lies in the creation of a functional prototype of the system, which can be used as a basis for the development of personal robotic assistants, as well as for integration into medical systems and educational and research laboratory testbeds.

Номер рядка	Формат	Позначення	Найменування	Кільк. аркушів	Номер екзем.	Примітка
1			<u>Документація загальна</u>			
2						
3			Знову розроблена			
4						
5	A4	ІК23.130БАК.006 ПЗ	Інтелектуальна система	71		
6			керування роботизованою			
7			рукою із застосуванням			
8			комп'ютерного зору.			
9			Пояснювальна записка.			
10						
11	A3	ІК23.130БАК.006 Е1	Інтелектуальна система	1		
12			керування роботизованою			
13			рукою із застосуванням			
14			комп'ютерного зору.			
15			Схема електричних з'єднань.			
16						
17	A3	ІК23.130БАК.006 Д1	Інтелектуальна система	1		
18			керування роботизованою			
19			рукою із застосуванням			
20			комп'ютерного зору.			
21			Блок-схема роботи системи.			
22						
23	A3	ІК23.130БАК.006 Д2	Інтелектуальна система	1		
24			керування роботизованою			
25			рукою із застосуванням			
26			комп'ютерного зору.			
27			Графіки роботи ЕМА-фільтрів.			
Зм.	Аркуш	№ докум.	Підпис	Дата	ІК23.130БАК.006 ТП Інтелектуальна система керування роботизованою рукою із застосуванням комп'ютерного зору. Відомість проєкту	
Розроб.		Павлов А. Т.				
Керівн.		Нікітін В. А.				
Затв.					Літ. Т Аркуш 1 Аркушів 4 КПІ ім. Ігоря Сікорського Група ІК-23	

Номер рядка	Формат	Позначення			Найменування			Кільк. аркушів	Номер екзем.	Примітка
28	A2	ІК23.130БАК.006 100 СБ			Інтелектуальна система			1		
29					керування роботизованою					
30					рукою із застосуванням					
31					комп'ютерного зору.					
32					Роботизований маніпулятор.					
33					Складальний кресленик.					
34										
35	A2	ІК23.130БАК.006 200 СБ			Інтелектуальна система			1		
36					керування роботизованою					
37					рукою із застосуванням					
38					комп'ютерного зору.					
39					Вузол вказівного пальця.					
40					Складальний кресленик.					
41										
42	A2	ІК23.130БАК.006 300 СБ			Інтелектуальна система			1		
43					керування роботизованою					
44					рукою із застосуванням					
45					комп'ютерного зору.					
46					Вузол великого пальця.					
47					Складальний кресленик.					
48										
49	A3	ІК23.130БАК.006 001			Інтелектуальна система			1		
50					керування роботизованою					
51					рукою із застосуванням					
52					комп'ютерного зору.					
53					Основа долоні.					
54										
55										
					ІК23.130БАК.006 ТП					Арк.
										2
Зм.	Аркуш	№ докум.	Підпис	Дата						

Номер рядка	Формат	Позначення			Найменування			Кільк. аркушів	Номер екзем.	Примітка
56	A3	IK23.130BAK.006 002			Інтелектуальна система			1		
57					керування роботизованою					
58					рукою із застосуванням					
59					комп'ютерного зору.					
60					Кришка долоні.					
61										
62	A3	IK23.130BAK.006 003			Інтелектуальна система			1		
63					керування роботизованою					
64					рукою із застосуванням					
65					комп'ютерного зору.					
66					Захисна накладка на руку.					
67										
68	A3	IK23.130BAK.006 004			Інтелектуальна система			1		
69					керування роботизованою					
70					рукою із застосуванням					
71					комп'ютерного зору.					
72					Основа пальців.					
73										
74	A3	IK23.130BAK.006 005			Інтелектуальна система			1		
75					керування роботизованою					
76					рукою із застосуванням					
77					комп'ютерного зору.					
78					Середня фаланга пальців.					
79										
80										
81										
82										
83										
					IK23.130BAK.006 ТП					Арк.
										3
Зм.	Аркуш	№ докум.	Підпис	Дата						

Номер рядка	Формат	Позначення	Найменування	Кільк. аркушів	Номер екзем.	Примітка	
84	A3	IK23.130БАК.006 006	Інтелектуальна система	1			
85			керування роботизованою				
86			рукою із застосуванням				
87			комп'ютерного зору.				
88			Дистальна фаланга пальців.				
89							
90	A3	IK23.130БАК.006 007	Інтелектуальна система	1			
91			керування роботизованою				
92			рукою із застосуванням				
93			комп'ютерного зору.				
94			Кінчик пальців.				
95							
96	A3	IK23.130БАК.006 008	Інтелектуальна система	1			
97			керування роботизованою				
98			рукою із застосуванням				
99			комп'ютерного зору.				
100			Поворотний шарнір				
101			великого пальця.				
102							
103	A3	IK23.130БАК.006 009	Інтелектуальна система	1			
104			керування роботизованою				
105			рукою із застосуванням				
106			комп'ютерного зору.				
107			Проміжна фаланга				
108			великого пальця.				
109							
110							
111							
			IK23.130БАК.006 ТП				Арк.
							4
Зм.	Аркуш	№ докум.					Підпис

**Пояснювальна записка
до дипломного проєкту
на тему: «Інтелектуальна система керування
роботизованою рукою із застосуванням
комп'ютерного зору»**

Київ – 2026 року

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	4
ВСТУП	5
1 ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ	8
1.1 Сфери застосування антропоморфних роботів-маніпуляторів у сучасній економіці	8
1.2 Загальні характеристики та особливості антропоморфних робототехнічних систем	9
1.3 Інтелектуальні методи керування роботизованими маніпуляторами	11
1.4 Постановка задачі	13
Висновки до розділу 1	14
2 АНАЛІЗ ІСНУЮЧИХ РІШЕНЬ ТА МЕТОДІВ КЕРУВАННЯ	15
2.1 Сучасні роботизовані маніпулятори	15
2.2 Аналіз існуючих систем жестового керування	16
2.2.1 Системи керування на основі маркерів та сенсорних пристроїв	16
2.2.2 Безмаркерні системи на основі комп'ютерного зору	17
2.2.3 Системи з використанням алгоритмів машинного навчання	17
2.3 Аналіз систем голосового керування	18
Висновки до розділу 2	19
3 ПРОЄКТУВАННЯ ІНТЕЛЕКТУАЛЬНОЇ СИСТЕМИ КЕРУВАННЯ	21
3.1 Обґрунтування підходу до реалізації системи жестового керування	21
3.2 Формування архітектури системи "зір – інтерпретація – керування"	22
3.3 Моделювання процесів взаємодії компонентів системи	23
3.4 Алгоритми інтерпретації жестових команд на основі MediaPipe	24
3.5 Метод перетворення координат у керуючі сигнали	27
3.6 Методи фільтрації даних та усунення шумів	28
3.7 Вимоги до умов функціонування системи	29

					ІК23.130БАК.006 ПЗ		
Зм.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Павлов А.Т.			Інтелектуальна система керування роботизованою рукою із застосуванням комп'ютерного зору. Пояснювальна записка	Лім.	Арк.
Перевір.		Нікітін В.А.				Т	2
						Акруїв	
						71	
Н. Контр.						КПІ ім. Ігоря Сікорського	
Затверд.						Група ІК-23	

Висновки до розділу 3	31
4 ВИБІР АПАРАТНО-ТЕХНІЧНИХ ТА ПРОГРАМНИХ ЗАСОБІВ	33
4.1 Обґрунтування вибору апаратної платформи	33
4.1.1 Конструкція роботизованого маніпулятора та ступені свободи	33
4.1.2 Вибір сервоприводів	35
4.1.3 Вибір мікроконтролера	36
4.1.4 Засоби отримання та обробки відеопотоку	37
4.2 Програмне забезпечення для розробки	38
4.2.1 Мова програмування Python	38
4.2.2 Середовища розробки	39
4.2.3 Застосування бібліотек MediaPipe, NumPy та PySerial	39
Висновки до розділу 4	41
5 РЕАЛІЗАЦІЯ ІНТЕЛЕКТУАЛЬНОЇ СИСТЕМИ КЕРУВАННЯ	42
5.1 Реалізація модуля комп'ютерного зору на основі MediaPipe	42
5.2 Реалізація алгоритмів обробки та класифікації жестів	43
5.3. Розробка системи керування роботизованим маніпулятором	45
5.4. Інтеграція програмних та апаратних компонентів	48
5.5 Організація обміну даними через PySerial	49
5.6 Оптимізація обчислень із NumPy	51
Висновки до розділу 5	52
6 ТЕСТУВАННЯ ІНТЕЛЕКТУАЛЬНОЇ СИСТЕМИ КЕРУВАННЯ	53
6.1 Методика тестування	53
6.2 Оцінка точності розпізнавання жестів	55
6.3 Аналіз часу відгуку та впливу затримок системи	58
6.4 Аналіз стабільності роботи та впливу шумів	59
6.5 Узагальнення отриманих результатів	62
Висновки до розділу 6	63
ВИСНОВКИ	65
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ	67
ДОДАТОК А Репозиторій проекту	71

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

EMA (EMA) — експоненціальне ковзне середнє (Exponential Moving Average)

ПЗ — програмне забезпечення

ПК — персональний комп'ютер

ШИМ (PWM) — широтно-імпульсна модуляція

CNN — згорткова нейронна мережа (Convolutional Neural Network)

FDM — моделювання методом наплавлення (Fused Deposition Modeling)

FPS — кількість кадрів на секунду (Frames Per Second)

I2C — послідовна асиметрична шина зв'язку (Inter-Integrated Circuit)

PETG — поліетилентерефталат-гліколь (жорсткий пластик для 3D-друку)

TPU — термопластичний поліуретан (гнучкий пластик для 3D-друку)

UART — універсальний асинхронний приймач-передавач (Universal Asynchronous Receiver-Transmitter)

					ІК23.130БАК.006 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		4

ВСТУП

Стрімкий розвиток робототехніки докорінно змінює підходи до виконання завдань, які ще донедавна вимагали виключно людської участі. Сьогодні автоматизовані системи виходять за межі жорстко регламентованого промислового виробництва і активно інтегруються в медицину, сферу обслуговування та роботу в небезпечних середовищах. Для виконання нетипових просторових завдань у цих галузях дедалі частіше застосовуються антропоморфні маніпулятори — електромеханічні системи, кінематика яких відтворює будову людської руки. Завдяки такій подібності вони володіють високою гнучкістю та здатні ефективно адаптуватися до нестандартних умов.

Однак розширення сфер застосування таких маніпуляторів виявляє суттєву проблему: складність взаємодії між людиною та машиною. Традиційні методи керування за допомогою громіздких пультів, джойстиків або попередньо запрограмованих траєкторій вимагають специфічних навичок від оператора, створюють високе когнітивне навантаження та обмежують швидкість реагування. Це суттєво ускладнює їх інтеграцію в середовища, де критично важливою є простота, оперативність та відсутність бар'єрів між діями людини та реакцією механізму. Саме тому в сучасній робототехніці виникає гостра потреба у розробці більш природних та інтуїтивних інтерфейсів керування [1,2].

З розвитком комп'ютерного зору та алгоритмів штучного інтелекту одним із найперспективніших рішень стає безконтактне жестове керування. Такий підхід дозволяє зчитувати рухи руки оператора за допомогою звичайної камери без використання додаткових дорогих чи громіздких сенсорних пристроїв і транслювати їх у керуючі сигнали для виконавчих механізмів маніпулятора в режимі реального часу. Окрім технологічної інноваційності, такі системи значно підвищують мобільність оператора та виводять людино-машинну взаємодію на новий рівень.

Метою роботи є підвищення інтуїтивності, точності та стабільності людино-машинної взаємодії шляхом розробки інтелектуальної системи керування антропоморфним роботизованим маніпулятором на основі комп'ютерного зору, що забезпечує точне відтворення рухів руки оператора в режимі реального часу.

Для досягнення поставленої мети визначено наступні завдання:

- проаналізувати предметну область, сфери застосування антропоморфних систем та дослідити існуючі методи керування (сенсорні, безмаркерні, голосові);
- спроектувати архітектуру інтелектуальної системи за моделлю «зір – інтерпретація – керування» та змодельовати процеси взаємодії її компонентів;
- обґрунтувати вибір апаратно-технічних платформ (маніпулятор, мікроконтролер) та програмних засобів для реалізації проєкту;
- розробити та програмно реалізувати алгоритми інтерпретації жестових команд на основі MediaPipe, методи двоступеневої фільтрації даних та перетворення просторових координат у керуючі сигнали (mapping);
- здійснити інтеграцію апаратних і програмних компонентів та організувати обмін даними через послідовний інтерфейс;
- провести комплексне тестування розробленої системи, оцінити точність розпізнавання жестів, час відгуку та стабільність роботи в умовах апаратних шумів.

Об'єктом роботи є процес взаємодії людини з роботизованим маніпулятором засобами безконтактного жестового керування.

Предметом роботи є методи комп'ютерного зору, алгоритми обчислення кутів суглобів та двоступеневої фільтрації сигналів, а також програмно-апаратні засоби перетворення координат у керуючі PWM-команди для сервоприводів маніпулятора.

Актуальність полягає в поширеній проблемі складності та обмеженості традиційних інтерфейсів керування робототехнічними системами.

Впровадження безконтактних рішень на основі комп'ютерного зору дає змогу спростити процес взаємодії оператора з робототехнічною системою, зменшити потребу в тривалому навчанні операторів, підвищити комфорт та оперативність керування, а також мінімізувати вплив апаратних шумів на точність виконання маніпуляцій.

Практична цінність проявляється в можливості реалізації робочого прототипу програмно-апаратного комплексу керування маніпулятором у режимі копіювання рухів. Розроблені алгоритми фільтрації сигналів, метод індивідуального калібрування сервоприводів та механізм обробки відеопотоку можуть бути використані як основа для створення персональних роботів-асистентів і медичних систем або впроваджені у сфери, де необхідне точне копіювання дрібної моторики людини на відстані.

Дипломний проєкт складається зі вступу, 6 розділів (опис предметної області, аналіз існуючих рішень, проєктування системи, вибір засобів реалізації, програмно-апаратна реалізація, тестування системи), висновків, переліку використаних джерел із 27 найменувань та додатків. Графічна частина включає 15 креслеників (або ілюстрацій/схем). Загальний обсяг пояснювальної записки становить 71 сторінок друкованого тексту.

1 ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Сфери застосування антропоморфних роботів-маніпуляторів у сучасній економіці

Антропоморфні маніпулятори не просто технічна цікавинка. Їхня конструкція, що відтворює кінематику людської руки з кількома ступенями свободи, дає їм одну ключову перевагу — універсальність [3]. Там, де вузькоспеціалізований промисловий робот справляється лише з однією операцією, антропоморфна система здатна адаптуватися до зовсім різних задач без суттєвої переробки конструкції. Саме це робить такі системи затребуваними одразу в кількох галузях [4].

Найбільш очевидна сфера — промислове виробництво. У машинобудуванні, автомобільній та електронній промисловості маніпулятори виконують зварювання, складання, фарбування та контроль якості [5]. Висока точність позиціонування й повторюваність рухів дозволяють суттєво знизити відсоток браку та зменшити вплив людського фактора на кінцевий результат.

У логістиці антропоморфні роботи вирішують завдання, з якими класичні конвеєрні системи справляються погано: сортування різнорідних товарів, пакування, комплектація замовлень. Коли асортимент продукції постійно змінюється — а в сучасній електронній комерції це норма — потрібна система, здатна працювати з об'єктами різної форми та розміру без перепрограмування під кожну нову позицію.

Медицина — окрема і дуже показова сфера. Реабілітаційні пристрої на основі маніпуляторів допомагають пацієнтам відновлювати рухову функцію після інсультів або травм — система забезпечує контрольовані, повторювані рухи з урахуванням індивідуального стану пацієнта. Антропоморфна конструкція тут є не естетичним рішенням, а функціональною необхідністю [6].

Сервісна сфера поки знаходиться на ранньому етапі впровадження, але тенденція очевидна.

					ІК23.130БАК.006 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		8

Роботи-помічники в готелях, автоматизовані системи обслуговування, доставка всередині будівель — усе це вже існує не лише як концепт [7]. Головна вимога тут — безпечна й природна взаємодія з людиною в непередбачуваному середовищі.

В освіті та науці маніпулятори відіграють роль універсальних дослідницьких платформ. На них відпрацьовуються алгоритми керування, перевіряються гіпотези у сфері комп'ютерного зору та машинного навчання, проводяться експерименти з інтерфейсами «людина–робот». Модульність і відносна доступність сучасних платформ зробили їх стандартним інструментом університетських лабораторій.

Сільське господарство — менш очевидна, але перспективна галузь [8]. Збір врожаю, сортування плодів, догляд за рослинами в теплицях — операції, які важко автоматизувати стандартними засобами через мінливість форм і розмірів об'єктів. Антропоморфні системи тут виглядають логічним рішенням, особливо в умовах хронічного дефіциту сезонної робочої сили.

Отже, сфери застосування антропоморфних маніпуляторів охоплюють широкий спектр сучасної економічної діяльності. Спільне між усіма цими галузями одне: потреба у гнучкій, точній та адаптивній системі, здатній виконувати дії, що раніше вважалися виключно людською прерогативою.

1.2 Загальні характеристики та особливості антропоморфних робототехнічних систем

Якщо говорити про те, що відрізняє антропоморфний маніпулятор від будь-якого іншого промислового робота — це насамперед кінематика. Конструкція таких систем частково відтворює анатомію людської руки: наявні плечовий сегмент, ліктьовий, кистьовий. Проте це не данина естетиці. Така будова дозволяє виконувати складні просторові траєкторії та змінювати орієнтацію робочого органу так, як це робить людина. Звідси й назва — антропоморфний, тобто людиноподібний.

Ключова технічна характеристика — кількість ступенів свободи. Чим їх більше, тим ширший діапазон рухів і тим складніші операції доступні системі [9].

Але є й зворотний бік: зі збільшенням ступенів свободи суттєво ускладнюються задачі керування. Потрібні ефективніші алгоритми планування траєкторій, більш потужна обчислювальна база і точніша обробка сенсорних даних. Це один із головних інженерних викликів у розробці таких систем.

Рухи окремих ланок маніпулятора забезпечують виконавчі механізми — найчастіше електричні сервоприводи. Вони поєднують достатню потужність із точністю позиціонування і добре інтегруються з мікроконтролерними платформами. Важливо, що сучасні сервоприводи підтримують зворотний зв'язок, що дає змогу будувати системи керування із замкненим контуром — тобто такі, що не просто виконують команду, а й контролюють її виконання.

Окремої уваги заслуговує сенсорна підсистема. Датчики положення, швидкості, прискорення — все це дає системі інформацію про власний стан. Але для взаємодії з навколишнім середовищем і, зокрема, з користувачем потрібно більше. Камери та алгоритми комп'ютерного зору дозволяють реалізувати безконтактне керування — наприклад, через розпізнавання жестів. Без розвиненої сенсорики побудувати справді адаптивну систему практично неможливо.

Програмна частина антропоморфних систем не менш складна, ніж механічна. Алгоритми прямої та зворотної кінематики, планувальники траєкторій, модулі обробки сенсорних даних, інтерфейси взаємодії з користувачем — усе це має працювати узгоджено і в реальному часі. Останніми роками сюди активно додаються методи машинного навчання, які підвищують автономність системи та її здатність пристосовуватися до змін середовища.

Важлива особливість антропоморфних систем — орієнтація на роботу поруч із людиною. Це накладає специфічні вимоги: обмеження швидкості й сили рухів, виявлення перешкод, розпізнавання намірів оператора.

Інтерфейс керування має бути не просто функціональним, а ергономічним: жести, голос або інші природні способи комунікації.

Варто також відзначити модульність як принцип побудови. Більшість сучасних антропоморфних платформ дозволяють змінювати кількість ступенів свободи, замінювати приводи або сенсори без переробки всієї конструкції. Це суттєво спрощує модернізацію та адаптацію під конкретні задачі.

Разом із тим існують і реальні обмеження. Математичний опис рухів багатоланкової системи — нетривіальна задача. Обробка великого потоку сенсорних даних у реальному часі вимагає відповідної обчислювальної потужності. А питання енергоспоживання стає критичним у мобільних або автономних конфігураціях, де немає постійного підключення до мережі.

Загалом антропоморфний маніпулятор — це не просто механізм із моторами. Це інтегрований комплекс, де механічна конструкція, електроніка та програмне забезпечення однаково важливі. І саме баланс між цими складовими визначає, наскільки система буде ефективною на практиці.

1.3 Інтелектуальні методи керування роботизованими маніпуляторами

Класичні підходи до керування роботами — жорстко задані алгоритми, точні математичні моделі, наперед прописані траєкторії — добре працюють у передбачуваних умовах. Але реальне середовище рідко буває передбачуваним. Освітлення змінюється, об'єкти зміщуються, користувачі поведуться по-різному. Саме тут на перший план виходять інтелектуальні методи — ті, що дозволяють системі адаптуватися, а не просто виконувати інструкції.

Базовим інструментом для реалізації подібних систем виступає машинне навчання. Алгоритми класифікації та регресії використовуються для розпізнавання образів, прогнозування станів і формування керуючих сигналів. У контексті жестового керування це означає, що система не просто отримує координати руки — вона навчається інтерпретувати їх як конкретні команди, причому з урахуванням варіативності рухів різних людей [10].

Нейронні мережі займають окреме місце в цьому арсеналі. Згорткові архітектури (CNN) ефективно обробляють зображення та відеопотоки — саме на них базуються сучасні системи розпізнавання жестів і об'єктів.

Рекурентні мережі (RNN) ефективно працюють із часовими послідовностями, що важливо для аналізу рухів. Глибоке навчання підвищує точність, але потребує значних обчислювальних ресурсів і якісних даних.

Нечітка логіка — дещо інший підхід. Замість чітких «так/ні» вона оперує поняттям часткової істинності, що дозволяє гнучкіше реагувати на невизначені або граничні ситуації. У задачах керування маніпуляторами нечіткі регулятори використовуються там, де потрібна плавність рухів і здатність адаптуватися до незначних змін у поведінці системи — без повного перепрограмування.

Підкріплювальне навчання цікаве іншою ідеєю: система сама знаходить оптимальну стратегію поведінки через взаємодію із середовищем, отримуючи «винагороду» за правильні дії. Теоретично це дуже потужний інструмент — особливо для задач захоплення об'єктів або уникнення перешкод. На практиці ж навчання займає багато часу, а перенесення результатів із симуляції на реальне залізо досі залишається нетривіальною задачею.

Комп'ютерний зір як метод керування заслуговує окремої згадки. Детекція ключових точок, сегментація зображень, трекінг об'єктів — усе це дає системі можливість «бачити» користувача та навколишнє середовище без будь-яких фізичних датчиків на тілі людини [11]. Поєднання комп'ютерного зору з іншими інтелектуальними методами дозволяє будувати комплексні системи, здатні працювати в режимі реального часу з прийнятною точністю.

Слід також зазначити, що не менш важливими є фільтрація та обробка сигналів — без цього будь-яка інтелектуальна система буде нестабільною.

Фільтри Калмана, експоненціальне згладжування та подібні алгоритми прибирають шуми із сенсорних даних і дають більш точну картину стану системи. Особливо це актуально в процесі роботи з відеопотоком, де навіть нерухома рука дає певний розкид координат від кадру до кадру.

На практиці жоден із цих методів не використовується ізольовано. Сучасні системи керування — це завжди певний гібрид: нейронні мережі для розпізнавання, класична кінематика для розрахунку траєкторій, фільтри для стабілізації сигналу [12]. Такий підхід дозволяє взяти краще від кожного методу і компенсувати їхні слабкі сторони.

1.4 Постановка задачі

Антропоморфні маніпулятори мають широкий потенціал застосування, інтелектуальні методи керування активно розвиваються — але між існуючими підходами і зручним повсякденним використанням досі є розрив. Традиційні інтерфейси вимагають підготовки, спеціалізовані сенсорні системи — дорогі й незручні. Потрібне рішення, яке було б одночасно доступним, природним і технічно надійним.

Саме це і є задачею даної роботи. Необхідно розробити систему жестового керування антропоморфним маніпулятором, яка працює виключно на основі відеопотоку зі звичайної камери — без маркерів, рукавичок та будь-яких додаткових пристроїв на тілі оператора. Користувач просто рухає рукою, система це бачить і передає відповідні команди на приводи маніпулятора в режимі реального часу.

Технічно це розбивається на кілька конкретних підзадач. Перша — отримання відеопотоку та виявлення ключових точок руки на кожному кадрі. Друга — інтерпретація цих точок: визначення положення кисті в просторі та розпізнавання дискретних жестів для керування захоплювачем. Третя, і найскладніша — перетворення отриманих координат у реальні кути повороту сервоприводів з урахуванням кінематичних обмежень маніпулятора.

Окремою проблемою є стабільність. Дані з камери завжди містять певний шум — навіть при нерухомій руці координати точок від кадру до кадру трохи «плавають». Якщо передавати ці дані на приводи без обробки, маніпулятор постійно тремтітиме. Тому система повинна включати методи фільтрації та згладжування сигналів, які прибирають високочастотні коливання без помітної затримки у відгуку.

Крім програмної частини, необхідно забезпечити надійну інтеграцію з апаратною платформою. Обчислювальне ядро на базі ПК і мікроконтролер ESP32S3 мають злагоджено обмінюватися даними через послідовний інтерфейс — так, щоб команди доходили вчасно і без втрат.

Нарешті, розроблена система має пройти експериментальне тестування. Потрібно оцінити точність розпізнавання жестів, час відгуку, стабільність роботи при різних умовах освітлення та швидкості рухів оператора. Результати тестування дозволять не лише підтвердити працездатність системи, а й виявити напрями для подальшого вдосконалення.

Таким чином, поставлена задача охоплює повний цикл — від захоплення відеопотоку до фізичного руху маніпулятора. Її вирішення вимагає поєднання методів комп'ютерного зору, математики керування та програмно-апаратної інтеграції в єдиний злагоджений комплекс.

Висновки до розділу 1

У першому розділі розглянуто предметну область роботи та сфери застосування антропоморфних маніпуляторів, зокрема промисловість, логістику, медицину, сервіс, освіту та сільське господарство.

Проаналізовано конструктивні особливості маніпуляторів: багатоланкову кінематичну структуру, кілька ступенів свободи, сервоприводи зі зворотним зв'язком і сенсорну підсистему. Показано, що зростання функціональності підвищує складність керування та потребує використання інтелектуальних методів обробки даних.

Розглянуто основні підходи до інтелектуального керування, серед яких машинне навчання, нейронні мережі, нечітка логіка, підкріплювальне навчання, комп'ютерний зір і методи фільтрації сигналів. Встановлено, що найефективнішими є комбіновані рішення.

На основі проведеного аналізу сформульовано задачу розробки системи жестового керування маніпулятором на базі комп'ютерного зору, яка працює в реальному часі, не потребує додаткових сенсорів і забезпечує надійне перетворення жестів у керуючі сигнали.

Матеріал розділу є теоретичною основою для подальшого проектування та реалізації системи.

2 АНАЛІЗ ІСНУЮЧИХ РІШЕНЬ ТА МЕТОДІВ КЕРУВАННЯ

2.1 Сучасні роботизовані маніпулятори

Ринок роботизованих маніпуляторів сьогодні досить різноманітний — від масивних промислових установок до компактних настільних платформ, які можна зібрати з доступних компонентів. Щоб обґрунтовано вибрати базу для розробки системи жестового керування, варто розібратися, що взагалі існує і чим відрізняються основні класи таких систем.

Левову частку ринку традиційно займають класичні промислові маніпулятори — рішення від KUKA, FANUC, ABB, Yaskawa. Зазвичай це системи з чотирма-шістьма ступенями свободи, розраховані на високошвидкісні й точні операції: зварювання, фрезерування, пакування.

Точність позиціонування і надійність у них справді вражають. Але є суттєві обмеження: висока вартість, замкнута архітектура програмного забезпечення і повна відсутність можливості просто «підключити» сторонній алгоритм керування. Інтегрувати туди комп'ютерний зір або власні алгоритми — задача нетривіальна навіть для досвідченого інженера.

Окремий і активно зростаючий клас — коботи, тобто колаборативні роботи. Universal Robots із лінійкою UR та Franka Emika — найбільш відомі представники. Їхня головна перевага — вбудовані датчики моменту, які дозволяють безпечно працювати поруч із людиною без огорожень. Програмуються вони значно простіше за промислових побратимів, часто методом ручного навчання. Але ціна залишається дуже високою — для прототипування або навчального стенду це явно надлишкове рішення.

Для дослідницьких задач, розробки нових інтерфейсів та освітніх платформ найбільш затребуваними стали малогабаритні настільні маніпулятори [13]. Серед комерційних варіантів — Dobot Magician, uArm Swift. Але справжній тренд — це open-source платформи: InMoov, EEZYbotARM, BCN3D Moveo.

Більшість із них друкуються на 3D-принтері і будуються на базі сервоприводів та мікроконтролерів Arduino або Raspberry Pi.

					ІК23.130БАК.006 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		15

Відкрита архітектура забезпечує повний контроль над кожним приводом і дає змогу інтегрувати додаткові сенсори, камери та власні алгоритми без обмежень з боку виробника.

Саме такі платформи є оптимальними для задач даної роботи. Використання промислового або колаборативного робота для тестування системи жестового керування — економічно та технічно надлишкове рішення. Натомість відкрита платформа на базі ESP32 забезпечує необхідну гнучкість, прямий доступ до приводів і повну свободу у виборі методів керування.

2.2 Аналіз існуючих систем жестового керування

Жестове керування як концепція існує вже давно, але підходи до його реалізації суттєво відрізняються. Одні системи потребують спеціального обладнання на тілі оператора, інші обходяться лише камерою. У кожного підходу є свої сильні сторони і свої обмеження — і щоб зробити обґрунтований вибір, варто розглянути їх детальніше.

2.2.1 Системи керування на основі маркерів та сенсорних пристроїв

Історично першими з'явилися саме контактні системи [14]. Найвідоміший приклад — «кіберрукавички» (Data Gloves): рукавички з тензометрами, гіроскопами та акселерометрами, які вимірюють кути згину пальців і просторову орієнтацію кисті. Ще один варіант — електроміографічні браслети на кшталт Myo Armband, що зчитують електричні сигнали м'язів передпліччя під час виконання жестів. До цього ж класу належать оптичні маркерні системи — Vicon, OptiTrack — де набір інфрачервоних камер відстежує світловідбивні маркери, закріплені на руці оператора.

Точність у таких систем справді висока, затримка мінімальна, стійкість до умов освітлення — теж на рівні. Але є очевидні недоліки. Спеціальне обладнання коштує дорого, обмежує свободу рухів і вимагає регулярного калібрування.

Про природність взаємодії тут говорити не доводиться — одягнути рукавичку з дротами перед тим, як щось зробити, саме по собі незручно.

Такі рішення виправдані для вузькоспеціалізованих лабораторних досліджень або захоплення рухів у кінематографі. Для повсякденного керування маніпулятором — надлишкові і ергономічно сумнівні.

2.2.2 Безмаркерні системи на основі комп'ютерного зору

Альтернатива — безконтактні системи, де рухи людини аналізуються виключно через камеру. Класичні підходи базуються на обробці RGB або RGB-D зображень. Серед відомих рішень — Microsoft Kinect, Intel RealSense, Leap Motion. Алгоритми тут зазвичай включають сегментацію за кольором шкіри, виділення контурів, віднімання фону і пошук опуклої оболонки для визначення кінчиків пальців.

Перевага очевидна: користувачеві нічого не потрібно надягати, що забезпечує максимальну свободу дій. Але класичний комп'ютерний зір виявився вкрай чутливим до умов середовища. Зміна природного освітлення протягом дня, наявність у кадрі предметів, схожих за текстурою чи кольором на людську шкіру, а також анатомічне самоперекриття пальців під час складних жестів — і точність розпізнавання падає до неприйнятної рівня. RGB-D камери, які додатково вимірюють глибину сцени, частково вирішують проблему відокремлення фону, проте вони мають обмежену дальність роботи, чутливі до сонячного світла і суттєво здорожчують систему.

Тобто природність взаємодії є, а стабільності — ні. Для реального повсякденного використання поза межами контрольованої лабораторії з ідеально виставленим світлом таких систем виявляється замало.

2.2.3 Системи з використанням алгоритмів машинного навчання

Сучасний етап розвитку жестового керування характеризується широким впровадженням методів глибокого навчання.

У таких системах згорткові нейронні мережі безпосередньо виділяють ключові точки руки на RGB-зображенні [15]. Це дає змогу автоматично виявляти складні просторові закономірності та підвищувати точність розпізнавання жестів. Найбільш відомими рішеннями у цій сфері є OpenPose та MediaPipe від Google.

MediaPipe Hands заслуговує окремої уваги. Система в реальному часі локалізує 21 точку кисті — суглоби та кінчики пальців — навіть на відеопотоці зі звичайної веб-камери. Навчання на великих датасетах дозволило моделі ігнорувати складний фон, адаптуватися до різного освітлення і коректно передбачати положення пальців навіть при частковому перекритті чи під час швидких динамічних рухів. При цьому обчислювальне навантаження оптимізоване під роботу на звичайному CPU — без необхідності у відеокарті, що робить технологію повністю кросплатформною.

Підсумовуючи: підхід на основі машинного навчання усуває головні недоліки обох попередніх класів. Не потрібне спеціальне обладнання, як у сенсорних системах. Немає чутливості до умов середовища, як у класичного комп'ютерного зору. Завдяки цим перевагам досягається максимальна доступність рішення для кінцевого користувача. Саме тому MediaPipe обрано основою для реалізації системи керування в даній роботі.

2.3 Аналіз систем голосового керування

Голосове керування — ще один природний спосіб взаємодії людини з машиною [16], і було б неправильно його не розглянути. Технологічно воно базується на розпізнаванні мовлення та обробці природної мови: акустичний сигнал перетворюється на текст, текст інтерпретується як команда.

Сучасні рішення поділяються на хмарні та локальні. Хмарні — Google Assistant, Amazon Alexa, Microsoft Azure Speech — показують найвищу точність розпізнавання, оскільки використовують потужні серверні ресурси. Але вони залежать від стабільного інтернет-з'єднання і мають помітну затримку.

Для більшості побутових задач це не критично, але для керування маніпулятором у реальному часі — вже проблема. Локальні рішення — Vosk, Whisper, CMU Sphinx — позбавлені цих недоліків, але вимагають значних апаратних ресурсів для запуску нормальних мовних моделей.

Беззаперечна перевага голосового інтерфейсу — руки оператора залишаються вільними. Крім того, голос ідеально підходить для передачі простих дискретних команд: «стоп», «початкове положення», «захопити». Такі команди легко формулюються словами і не потребують точного просторового позиціонування.

Але саме тут і починається головна проблема. Маніпулятор рухається у тривимірному просторі — і цей рух за своєю природою є безперервним. Голосом описати плавну просторову траєкторію практично неможливо. Спробуйте словами передати «підніми руку на п'ять сантиметрів і одночасно поверни кисть на п'ятнадцять градусів вліво» — і стане зрозуміло, наскільки це незручно порівняно зі звичайним жестом.

Є й інша проблема — акустичні шуми. Виробниче або лабораторне середовище рідко буває тихим: мотори, вентиляція, сторонні розмови — все це суттєво знижує точність розпізнавання. На відміну від голосу, жестове керування до звукового середовища байдуже.

Отже, голосовий інтерфейс і жестове керування — не конкуренти, а скоріше доповнення одне до одного. Голос зручний для високорівневих команд, жест — для точного просторового керування. У межах даної роботи, де головна задача — забезпечити плавне і точне управління приводами маніпулятора, комп'ютерний зір і розпізнавання жестів є значно більш відповідним підходом.

Висновки до розділу 2

Проведений аналіз дозволяє зробити кілька практичних висновків, які безпосередньо впливають на вибір підходу до реалізації системи.

Щодо апаратної бази — промислові маніпулятори і коботи відпадають одразу: закрыта архітектура, висока вартість і складність інтеграції роблять їх

					ІК23.130БАК.006 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		19

непридатними для прототипування нових інтерфейсів керування. Відкриті платформи на базі ESP32S3 — логічний і обґрунтований вибір. Прямий доступ до приводів, гнучкість у налаштуванні, доступна елементна база — усе це робить їх оптимальними для задач даної роботи.

Щодо методів жестового керування картина теж досить чітка. Контактні системи — рукавички, маркери, ЕМГ-браслети — забезпечують високу точність, але обмежують рухи оператора і незручні в повсякденній експлуатації. Класичний комп'ютерний зір знімає проблему контакту, але нестабільно працює при змінному освітленні та складному фоні. Підхід на основі машинного навчання — зокрема MediaPipe — усуває обидва недоліки: не потребує додаткового обладнання і стійко працює в реальних умовах.

Голосове керування, незважаючи на зручність для простих команд, не підходить як основний інтерфейс для просторового керування маніпулятором. Дискретна природа мовлення і чутливість до акустичних шумів роблять його непридатним для точного і безперервного позиціонування.

Таким чином, для реалізації системи обрано такий підхід: апаратна платформа на базі ESP32S3 з відкритою архітектурою та метод жестового керування на основі нейромережевого трекінгу ключових точок руки через MediaPipe. Цей вибір підкріплений як технічними аргументами, так і практичними міркуваннями — доступністю, гнучкістю і реальною придатністю до роботи поза лабораторними умовами. Саме на цій основі будується архітектура системи, що розглядається в наступному розділі.

3 ПРОЄКТУВАННЯ ІНТЕЛЕКТУАЛЬНОЇ СИСТЕМИ КЕРУВАННЯ

3.1 Обґрунтування підходу до реалізації системи жестового керування

Концептуальна основа системи — режим тіні. Роботизована рука не просто реагує на окремі жести-команди, вона в реальному часі копіює положення пальців оператора: кожен згин, кожне випрямлення одразу відтворюється відповідним сервоприводом маніпулятора. Людська рука — дзеркало, а робот — відображення.

При цьому система розрізняє, яка рука знаходиться в кадрі. Якщо оператор показує ліву руку — маніпулятор відтворює рухи з лівосторонньою орієнтацією, якщо праву — з правосторонньою. Це важливо: кутові значення для сервоприводів при зміні руки інвертуються, щоб зберегти природність відображення і не порушити механіку великого пальця, який має різну базову орієнтацію залежно від руки. Реалізація такого підходу будується на трьох принципах.

Обробка відеопотоку і нейромережевий інференс — ресурсомісткі задачі, які виконуються на стороні ПК. Мікроконтролер ESP32S3 отримує вже готові значення кутів і займається виключно керуванням сервоприводами.

Замість абстрактного «жест → команда» система обчислює реальні кути згину в кожному суглобі пальця оператора і передає їх на відповідні канали маніпулятора. 16 каналів — 16 ступенів свободи. Результат — маніпулятор відтворює положення руки так само, як тінь повторює рухи тіла.

Нейромережева модель забезпечує стабільне відстеження 21 ключової точки кисті в реальному часі навіть на звичайній веб-камері. Це знімає необхідність у будь-якому додатковому обладнанні і дає достатньо даних для точного обчислення кутів суглобів. Завдяки такій синергії алгоритмів та механіки затримка стає майже невідчутною, а роботизована рука перетворюється на природне продовження людського тіла, яке не потребує навчання для керування.

3.2 Формування архітектури системи "зір – інтерпретація – керування"

Архітектура системи побудована як однонаправлений конвеєр: від кадру камери — до фізичного руху сервопривода. Кожен етап має чітко визначену функцію і не втручається в роботу сусіднього. Весь конвеєр розбито на чотири модулі.

OpenCV ініціалізує камеру і зчитує кадри в безперервному циклі. Кожен кадр одразу віддзеркалюється по горизонталі — це створює ефект дзеркала і робить керування інтуїтивним. Після цього зображення конвертується з BGR у RGB, оскільки MediaPipe працює саме з таким форматом.

Найскладніший і найважливіший модуль. Працює асинхронно: MediaPipe отримує кадр через `detect_async` і повертає результат у `callback`-функцію, не блокуючи головний цикл. Всередині модуля послідовно виконується кілька операцій.

Спочатку визначається рука в кадрі — ліва чи права. Через те що відео віддзеркалене, мітки MediaPipe інвертуються: «Left» від моделі насправді відповідає правій руці оператора. Далі перевіряється орієнтація долоні — чи повернута вона до камери чи тильною стороною. Якщо рука перевернута, координати по осі x відображаються дзеркально, щоб кути суглобів розраховувались коректно незалежно від повороту.

Після цього до 3D-координат точок застосовується експоненціальне згладжування з коефіцієнтом $\alpha=0.5$ — це прибирає дрібні стрибки між кадрами ще до обчислення кутів. Потім для кожного суглоба кожного пальця розраховується кут через три сусідні точки і функцію скалярного добутку.

Всього — 15 кутів плюс базовий канал великого пальця, разом 16 значень для 16 каналів маніпулятора.

Паралельно скелет кисті відображається на окреме полотно, яке у версії з CNN подається на вхід нейромережі для розпізнавання статичного жесту.

Розраховані кути пакуються у текстові команди формату `A:{канал}:{кут}\n` і відправляються через Serial-порт зі швидкістю 115200 бод. Команди надсилаються

не на кожен кадр, а з обмеженням — не частіше ніж раз на 20 мс, що запобігає переповненню буфера мікроконтролера.

На стороні ESP32S3 встановлено драйвер PCA9685, який керує 16 каналами сервоприводів через I2C. Мікроконтролер безперервно зчитує Serial-буфер, парсить команди і оновлює цільові значення PWM для кожного каналу. Фізичний рух відбувається не миттєво — на ESP32S3 реалізовано власний алгоритм ЕМА з коефіцієнтом $\alpha=0.15$, який оновлюється кожні 10 мс. Це забезпечує плавність рухів на апаратному рівні. Виняток — базовий канал великого пальця: він перемикається миттєво, оскільки відповідає за зміну орієнтації при зміні руки.

Розподіл між Python на ПК і C++ на мікроконтролері тут не випадковий. Python забезпечує гнучкість і зручність роботи з неймережами, C++ — передбачуване і швидке апаратне керування без накладних витрат.

3.3 Моделювання процесів взаємодії компонентів системи

Робота системи починається задовго до того, як оператор підніме руку перед камерою. Спочатку — підготовча фаза, і лише після неї запускається головний цикл.

На стороні ESP32S3: всі сервоприводи переводяться у початкове положення через драйвер PCA9685, Serial-порт відкривається на швидкості 115200 бод.

На стороні ПК: завантажується неймережева модель MediaPipe у оперативну пам'ять, ініціалізується камера, встановлюється Serial-з'єднання з мікроконтролером. Якщо підключена CNN — завантажується і вона. Тільки після успішного завершення всіх кроків система переходить у робочий режим.

Кожна ітерація — це повний шлях від пікселя до руху сервопривода. Камера віддає черговий кадр. Він одразу віддзеркалюється, конвертується у RGB і асинхронно передається до MediaPipe через `detect_async`. Результат надходить у `callback` і зберігається у змінній — головний цикл не чекає відповіді і не блокується. На наступній ітерації цикл перевіряє, чи є результат. Якщо руку не виявлено — маніпулятор залишається у поточному положенні, цикл продовжується. Якщо рука є — починається обробка.

Спочатку визначається, яка рука в кадрі і чи не перевернута вона тильною стороною. Якщо перевернута — координати по осі x коригуються дзеркально. Після цього до 3D-точок застосовується ЕМА-згладжування з $\alpha=0.5$, що прибирає міжкадрові стрибки ще до розрахунку кутів.

Далі для кожного суглоба обчислюється кут через три сусідні точки. 15 кутів пальців плюс базовий канал великого пальця — разом 16 значень. Якщо при цьому виявляється зміна руки в кадрі, активується механізм плавного переходу: великий палець спочатку складається у безпечне положення, система витримує паузу близько двох секунд для завершення механічного руху, і лише потім перемикає базову орієнтацію.

Готові кути пакуються у команди формату $A:\{\text{канал}\}:\{\text{кут}\}$ і відправляються через Serial — але не на кожен кадр, а з обмеженням у 20 мс між відправками. Це запобігає переповненню буфера мікроконтролера.

На стороні ESP32S3 все відбувається паралельно і незалежно. Мікроконтролер безперервно слухає Serial-буфер, парсить команди і оновлює цільові значення PWM.

Кожні 10 мс запускається окремий цикл ЕМА-згладжування з $\alpha=0.15$ — поточне значення PWM плавно наближається до цільового. Саме тут і народжується плавний рух маніпулятора. Виняток — канал 6 великого пальця: він перемикається миттєво, без згладжування. Інтерфейс оновлюється на кожній ітерації: відеокадр із намальованим скелетом і окреме полотно з 3D-проекцією руки відображаються у двох вікнах програми.

3.4 Алгоритми інтерпретації жестових команд на основі MediaPipe

Основним завданням підсистеми комп'ютерного зору є перетворення неструктурованого відеопотоку на набір формалізованих кінематичних параметрів кисті оператора [17]. Для реалізації цього завдання використано нейромережевий фреймворк MediaPipe (модуль HandLandmarker). На відміну від класичних систем, які використовують класифікацію кінцевого набору статичних жестів, запропонована система функціонує в «режимі тіні».

Це вимагає безперервного обчислення кутів згину кожного суглоба в просторі.

Процес інтерпретації просторових даних розбито на кілька послідовних алгоритмічних етапів:

На кожному кадрі вхідного відеопотоку нейромережа виконує інференс і повертає масив із 21 ключової точки (landmarks). Точки формують топологічну модель кисті: P0 відповідає зап'ястю, P1 – P4 — фалангам великого пальця, а наступні четвірки точок (P5 – P8, P9 – P12 тощо) — вказівному, середньому, безіменному пальцям та мізинцю відповідно.

Кожна точка P_i ($i \in [0, 20]$) описується тривимірним вектором нормалізованих координат (див. формулу 3.1):

$$P_i = (x_i, y_i, z_i) \quad (3.1)$$

де x_i та y_i нормалізовані відносно ширини та висоти кадру (значення від 0.0 до 1.0), а z_i відображає глибину точки відносно зап'ястя (чим менше значення, тим ближче точка до камери).

Оскільки для інтуїтивного керування відеопотік програмно віддзеркалюється по горизонталі (ефект дзеркала), алгоритм виконує інверсію метаданих типу руки (Handedness), щоб «Left» від MediaPipe коректно розпізнавався як права рука оператора, і навпаки.

Критично важливим етапом є визначення орієнтації долоні (тильною чи внутрішньою стороною до камери). Алгоритм порівнює координати по осі X для основи великого пальця (P2) та основи мізинця (P17). Якщо рука перевернута, застосовується алгоритм дзеркального відображення координат x відносно геометричного центру кисті (див. формулу 3.2):

$$x'_i = 2 \times x_{center} - x_i \quad (3.2)$$

Це гарантує, що подальші розрахунки кутів суглобів залишатимуться коректними та математично узгодженими незалежно від того, як користувач повернув руку.

Через дискретну природу матриці камери та мікрозміни освітлення «сирі» координати MediaPipe містять високочастотний шум. Обчислення кутів безпосередньо з цих даних призводить до ефекту тремтіння. Тому ще до тригонометричних розрахунків до масиву 3D-точок застосовується експоненціальне згладжування (ЕМА) з коефіцієнтом $\alpha = 0.5$ (див. формулу 3.3):

$$P_t = \alpha \times P_{raw} + (1 - \alpha) \times P_{t-1} \quad (3.3)$$

Такий підхід дозволяє стабілізувати положення вершин векторів у просторі, зберігаючи при цьому швидкість реакції системи на рівні обробки відеокадру.

Після стабілізації просторових координат система переходить до розрахунку кінематики. Кут згину в кожному суглобі обчислюється на основі координат трьох суміжних точок: проксимальної P_A , центральної (суглоб) P_B та дистальної P_C .

Алгоритм буде два просторові вектори, що виходять із суглоба (див. формули 3.4 та 3.5):

$$v_1 = P_A - P_B \quad (3.4)$$

$$v_2 = P_C - P_B \quad (3.5)$$

Кут θ між фалангами знаходиться через скалярний добуток цих векторів, поділений на добуток їхніх довжин (див. формули 3.6 та 3.7):

$$\cos(\theta) = \frac{v_1 \times v_2}{\|v_1\| \|v_2\|} \quad (3.6)$$

$$\theta = \arccos(\cos(\theta)) \times \frac{180}{\pi} \quad (3.7)$$

Для уникнення помилок типу "Not a Number" (NaN) при програмних обчисленнях з рухомою комою, значення косинуса жорстко обмежується (clamping) у діапазоні $[-1.0, 1.0]$. Отриманий кут у градусах також обмежується в анатомічно допустимому діапазоні $[90^\circ, 180^\circ]$, де 90° — повністю зігнутий палець, а 180° — повністю випрямлений.

Процес векторного обчислення повторюється ітеративно для 15 рухомих суглобів (по 3 на кожен палець). Для великого пальця додатково обчислюється кут базової орієнтації, що визначає його відведення від долоні.

У результаті багатовимірний масив точок трансформується у вектор із 16 скалярних значень. Цей детермінований набір повністю описує поточний стан людської кисті та передається на наступний етап конвеєра — систему мапінгу (перетворення координат у керуючі ШІМ-сигнали для виконавчих механізмів).

3.5 Метод перетворення координат у керуючі сигнали

Між кутом суглоба, отриманим від MediaPipe, і реальним положенням сервопривода — два послідовних перетворення. Перше відбувається на стороні Python, друге — на ESP32S3. Разом вони утворюють повний ланцюг відображення від руху людини до руху машини.

Далі MediaPipe повертає кути суглобів у діапазоні $[90^\circ, 180^\circ]$, де 90° — повністю зігнутий палець, 180° — повністю випрямлений. ESP32S3 очікує ціле число від 0 до 180. Перетворення між цими діапазонами залежить від того, яка рука в кадрі.

Для правої руки (див. формулу 3.8):

$$\Theta_{esp} = \Theta_{mp} - 90 \quad (3.8)$$

Для лівої руки значення інвертується, щоб зберегти природність відображення (див. формулу 3.9):

$$\Theta_{esp} = 270 - \Theta_{mp} \quad (3.9)$$

Канал 7 — базовий суглоб великого пальця — має окрему формулу з механічним коефіцієнтом компенсації (див. формулу 3.10):

$$\Theta_{esp} = (\Theta_{mp} - 90) \times 3 \quad (3.10)$$

Це пов'язано з конструктивною особливістю приводу: через передаточне відношення редуктора для досягнення потрібного кута повороту необхідний втричі більший вхідний сигнал. Канал 6 — основа великого пальця — не обчислюється взагалі: він фіксується у значенні 000 для правої руки і 180 для лівої, оскільки відповідає за просторову орієнтацію пальця, а не за його згин.

До всіх отриманих значень застосовується обмеження (див. формулу 3.11):

$$\Theta_{out} = \max(0, \min(\Theta_{esp}, 180))$$

(3.11)

Кожен із 16 сервоприводів має індивідуально відкалібровані значення: мінімальний, середній і максимальний PWM. Це необхідно через механічні відмінності між окремими екземплярами сервоприводів. Перетворення кута у PWM відбувається кусково-лінійно (див. формули 3.12 та 3.13):

$$PWM = \text{map}(\Theta, 0, 90, PWM_{min}, PWM_{mid}), \Theta \leq 90 \quad (3.12)$$

$$PWM = \text{map}(\Theta, 90, 180, PWM_{mid}, PWM_{max}), \Theta > 90 \quad (3.13)$$

Отримане значення PWM стає цільовим для алгоритму ЕМА на ESP32S3, який плавно наближає поточний стан сервопривода до цільового кожні 10 мс.

Таким чином, весь ланцюг перетворення — від нормалізованих координат MediaPipe до мікросекунд PWM-сигналу — є детермінованим, швидким і враховує як геометрію людської руки, так і механічні особливості кожного окремого привода маніпулятора [18].

3.6 Методи фільтрації даних та усунення шумів

Шум у системі виникає на двох різних рівнях — і боротися з ним потрібно на кожному окремо. Саме тому фільтрація реалізована двоступенево: спочатку на стороні Python, потім на стороні ESP32S3.

Перший рівень — згладжування 3D-координат. Навіть при абсолютно нерухомій руці MediaPipe від кадру до кадру повертає дещо різні координати ключових точок. Причина — дискретність матриці камери, мікрозміни освітлення, особливості роботи нейромережевого оптимізатора. Якщо обчислювати кути суглобів із таких «сирих» даних — значення будуть стрибати, і маніпулятор тремтітиме навіть у статичному положенні.

Щоб усунути це, до масиву 3D-координат 21 точки застосовується ЕМА-згладжування з коефіцієнтом $\alpha=0.5$ до розрахунку кутів (див. формулу 3.14):

$$\vec{P}_t = \alpha \times \vec{P}_{raw} + (1 - \alpha) \times \vec{P}_{t-1} \quad (3.14)$$

Коефіцієнт 0.5 обраний як баланс між швидкістю реакції та плавністю: система достатньо швидко реагує на реальні рухи і водночас не пропускає дрібні шуми далі по конвеєру. При зміні орієнтації руки або першій появі в кадрі буфер скидається, щоб уникнути артефактів від попереднього стану.

Другий рівень — згладжування PWM на мікроконтролері. Навіть після першого рівня фільтрації розраховані кути можуть містити залишкові коливання. Крім того, механічна інерція сервоприводів сама по собі вимагає плавного підведення до цільового положення — різкі стрибки команд перевантажують редуктори і спричиняють характерне клацання.

На ESP32S3 реалізовано власний ЕМА-цикл із коефіцієнтом $\alpha=0.15$, що оновлюється кожні 10 мс (див. формулу 3.15):

$$PWM_t = \alpha \times PWM_{target} + (1 - \alpha) \times PWM_{t-1} \quad (3.15)$$

Менший коефіцієнт порівняно з Python-рівнем пояснюється тим, що тут важлива максимальна плавність фізичного руху, а не швидкість реакції — за реакцію вже відповів перший рівень. Якщо різниця між поточним і цільовим значенням стає меншою за 1.0 — сервопривод миттєво фіксується у цільовому положенні, що запобігає нескінченному наближенню з мінімальними коливаннями. Виняток — канал 6, базова орієнтація великого пальця. Він перемикається миттєво, без жодного згладжування, оскільки є структурним перемиканням між двома фіксованими позиціями, а не плавним рухом.

Комбінація двох рівнів фільтрації — координатного на Python і PWM-рівня на ESP32S3 — забезпечує стабільну роботу маніпулятора як під час активних рухів, так і у статичних положеннях, де тремтіння особливо помітне і шкідливе для механіки.

3.7 Вимоги до умов функціонування системи

Нейромережеві алгоритми MediaPipe суттєво стійкіші до зовнішніх умов порівняно з класичним комп'ютерним зором — але це не означає, що умови середовища не мають значення. Для стабільної роботи всього комплексу існує ряд практичних вимог

					ІК23.130БАК.006 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		29

Освітлення: це найкритичніший фактор. Недостатнє освітлення змушує камеру збільшувати витримку, через що рухи руки розмиваються на зображенні — і нейромережа або втрачає руку, або повертає нестабільні координати. Рекомендований рівень освітленості робочої зони — від 300 люкс. Контрове освітлення, коли джерело світла знаходиться прямо за рукою оператора, перетворює кисть на темний силует — у такому випадку детекція суглобів стає ненадійною.

Робоча дистанція і позиціонування: оптимальна відстань від камери до руки — від 0.4 до 1.2 метра. Якщо рука надто близько — пальці виходять за межі кадру при широких рухах. Якщо надто далеко — на кисть припадає мало пікселів, і точність визначення дрібних рухів пальців падає. Камера має бути зафіксована нерухомо і спрямована фронтально на оператора — будь-яке хитання камери сприймається системою як рух руки і вносить шум у керуючі сигнали.

Обчислювальні ресурси: оля забезпечення 25–30 ітерацій циклу за секунду ПК повинен мати достатній процесор для запуску MediaPipe у реальному часі. При використанні CNN-модуля навантаження зростає — у такому випадку бажана наявність дискретної відеокарти або достатньо потужного CPU. Нижче 15 FPS керування стає помітно менш плавним і природним.

Енергозабезпечення маніпулятора: одночасний рух кількох сервоприводів під навантаженням дає значні стрибки струму. Живлення логічної частини ESP32S3 від USB є прийнятним, але сервоприводи обов'язково мають отримувати живлення від окремого зовнішнього джерела — мінімум 5В з силою струму від 15А.

Недотримання цього правила призводить до просідання напруги, перезавантаження мікроконтролера і некерованих рухів маніпулятора.

Serial-з'єднання: кабеля між ПК, ESP32S3 і рукою мають бути надійно з'єднані — будь-яке переривання зв'язку під час роботи призводить до зупинки маніпулятора у поточному положенні. Швидкість 115200 бод є оптимальною для

даного обсягу даних і не вимагає додаткових налаштувань на стандартному обладнанні.

Висновки до розділу 3

У третьому розділі здійснено теоретичне проєктування та сформовано інформаційно-алгоритмічне підґрунтя для функціонування системи жестового керування роботизованим маніпулятором. Описано концепцію функціонування комплексу у режимі «тіні», який передбачає безперервне та синхронне відтворення кінематики людської кисті на основі динамічного обчислення просторових кутів згину суглобів оператора.

Обґрунтовано доцільність застосування розподіленої схеми обчислень, у межах якої ресурсомісткі процеси обробки відеопотоку й нейромережевого інференсу покладено на персональний комп'ютер, тоді як мікроконтролер ESP32S3 виконує локальні задачі низькорівневого формування керуючих сигналів для сервоприводів. Такий підхід дозволяє досягти оптимального балансу між обчислювальною гнучкістю алгоритмів штучного інтелекту та апаратною швидкодією контуру керування в режимі реального часу.

Побудовано функціональну модель конвеєра обробки даних за парадигмою «зір – інтерпретація – керування». Визначено чітку послідовність інформаційних перетворень, що охоплює асинхронне захоплення відеопотоку, виділення топологічної моделі руки з 21 ключової точки за допомогою засобів MediaPipe, векторно-математичний розрахунок міжфалангових кутів та їх подальшу трансформацію у вихідні ШІМ-команди.

Особливу увагу приділено методам підвищення стабільності та природності рухів механічної платформи.

Запропоновано та математично обґрунтовано схему двоступеневої фільтрації сигналів для компенсації апаратних шумів камери та детектора. Визначено параметри експоненціального згладжування (ЕМА) як для просторових 3D-координат точок на рівні програмного ядра Python, так і для вихідних PWM-сигналів на рівні мікроконтролера. Це створює теоретичну основу для зменшення ефекту тремтіння (jitter) приводів маніпулятора.

Розроблено математичний апарат маппінгу кутових координат суглобів із цільовими станами виконавчих механізмів.

Описаний алгоритм враховує анатомічні обмеження рухливості людської кисті, специфіку кінематики великого пальця, необхідність координатної інверсії для забезпечення інтуїтивності при зміні лівої та правої рук оператора, а також підтримує можливість кусково-лінійної адаптації під індивідуальні калібрувальні параметри сервоприводів.

Додатково сформульовано технічні вимоги до зовнішніх умов функціонування розробленого комплексу, зокрема щодо мінімального рівня й напрямку освітленості робочої зони, просторової дистанції до камери, обмеження частоти дискретизації при передачі пакетів даних та параметрів роздільного енергозабезпечення логічної й силової частин маніпулятора.

У результаті виконаного проєктування сформовано комплексне алгоритмічне, функціональне та математичне забезпечення системи безконтактного жестового керування. Синергія методів комп'ютерного зору, просторової геометрії та вбудованих систем дозволяє забезпечити детермінованість, високу плавність та мінімальну латентність інформаційного конвеєра, що є основою для подальшої практичної реалізації та тестування комплексу.

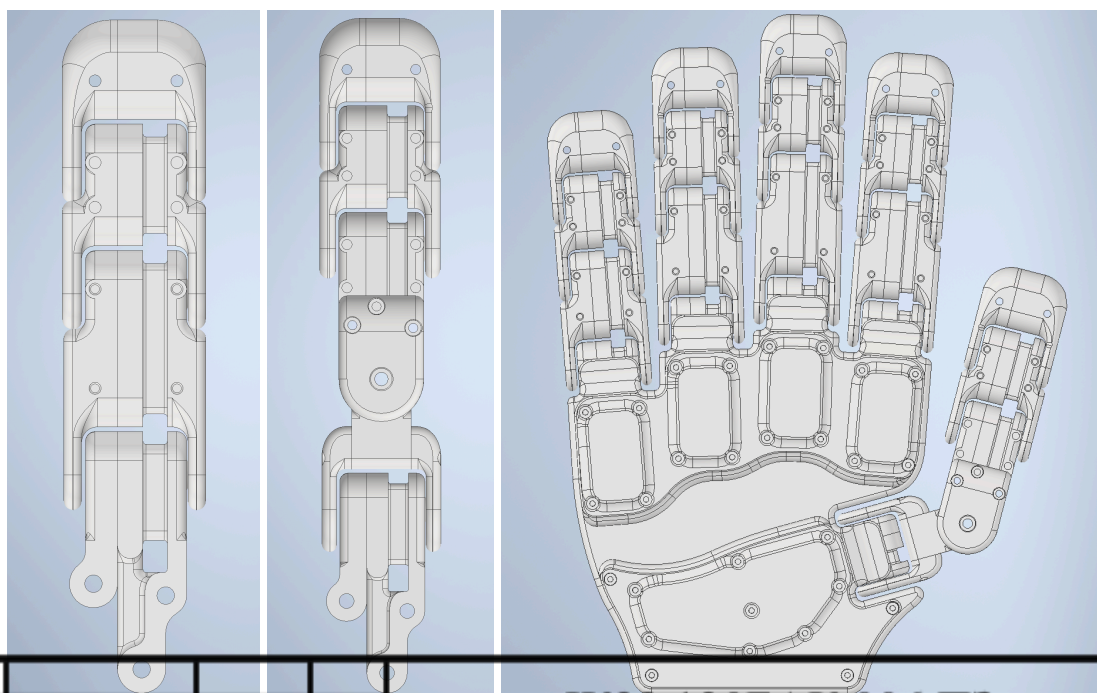
4 ВИБІР АПАРАТНО-ТЕХНІЧНИХ ТА ПРОГРАМНИХ ЗАСОБІВ

4.1 Обґрунтування вибору апаратної платформи

Вибір апаратної бази — один із ключових етапів розробки будь-якої робототехнічної системи. Від того, наскільки грамотно підібрані компоненти, залежить не лише функціональність кінцевого пристрою, але й можливість його подальшого розвитку та адаптації. У даному проєкті апаратна платформа формується з кількох взаємопов'язаних складових — механічної конструкції, виконавчих механізмів, мікроконтролера та системи технічного зору.

4.1.1 Конструкція роботизованого маніпулятора та ступені свободи

За основу взято антропоморфну роботизовану руку з п'ятьма пальцями, що відтворює кінематику людської кисті. Тривимірна модель конструкції розроблена у середовищі Autodesk Inventor 2024 — професійному CAD-інструменті, який дозволяє проєктувати складальні вузли з урахуванням кінематичних зв'язків між деталями та перевіряти геометричні обмеження ще на етапі проєктування. На рисунку 4.1 зображено компоненти маніпулятора в Autodesk Inventor.



ІК23.130БАК.006 ПЗ

Рисунок 4.1 — 3D-модель маніпулятора в Autodesk Inventor

Складальний кресленик усього роботизованого маніпулятора наведено на кресленику ІК23.130БАК.006 100 СБ, де відображено загальну компоновку та просторову взаємодію всіх елементів системи. Для деталізації складних рухомих механізмів розроблено складальний кресленик вузла вказівного пальця (наведено на кресленику ІК23.130БАК.006 200 СБ), який ілюструє шарнірні з'єднання фаланг, та складальний кресленик вузла великого пальця (наведено на кресленику ІК23.130БАК.006 300 СБ), що демонструє специфіку його поворотної бази та кінематичні особливості відведення. Робочі кресленики кожної окремої деталі розроблено з урахуванням технологічних допусків для точного відтворення їхньої комплексної геометрії під час наступного безлюфтового складання вузлів. Конструкцію основи долоні наведено на кресленику ІК23.130БАК.006 001, а її кришку — на кресленику ІК23.130БАК.006 002; разом вони утворюють захищену внутрішню каверну для прокладання кабельних комунікацій та елементів натягу приводів. Захисну накладку на руку наведено на кресленику ІК23.130БАК.006 003. Окремі елементи пальців, а саме основу, середню та дистальну фаланги, наведено на креслениках ІК23.130БАК.006 004, ІК23.130БАК.006 005 та ІК23.130БАК.006 006 відповідно. Кінчик пальців наведено на кресленику ІК23.130БАК.006 007. Деталі специфічної кінематики великого пальця, зокрема поворотний шарнір та проміжну фалангу, наведено на креслениках ІК23.130БАК.006 008 та ІК23.130БАК.006 009.

Усі деталі конструкції виготовлені методом FDM 3D-друку з двох різномірних матеріалів. Жорсткі несучі елементи (фаланги пальців, долоня, кріпильні вузли) надруковані з міцного полімеру PETG, який забезпечує високу жорсткість та стійкість до механічних деформацій при мінімальній вазі корпусу. Гнучкі елементи (накладки на долоні і пальцях) виконані з еластичного матеріалу TPU, що має необхідну пружність для тактильності, покращення фрикційних властивостей під час безпосереднього зчеплення комплексу з об'єктами маніпуляцій та ефективного гасіння вібраційних навантажень у статиці. На рисунку 4.2 зображено готову роботизовану руку в двох різних позиціях, що

					ІК23.130БАК.006 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		34

демонструють фінальний результат практичного втілення розробленого комплекту конструкторської документації.

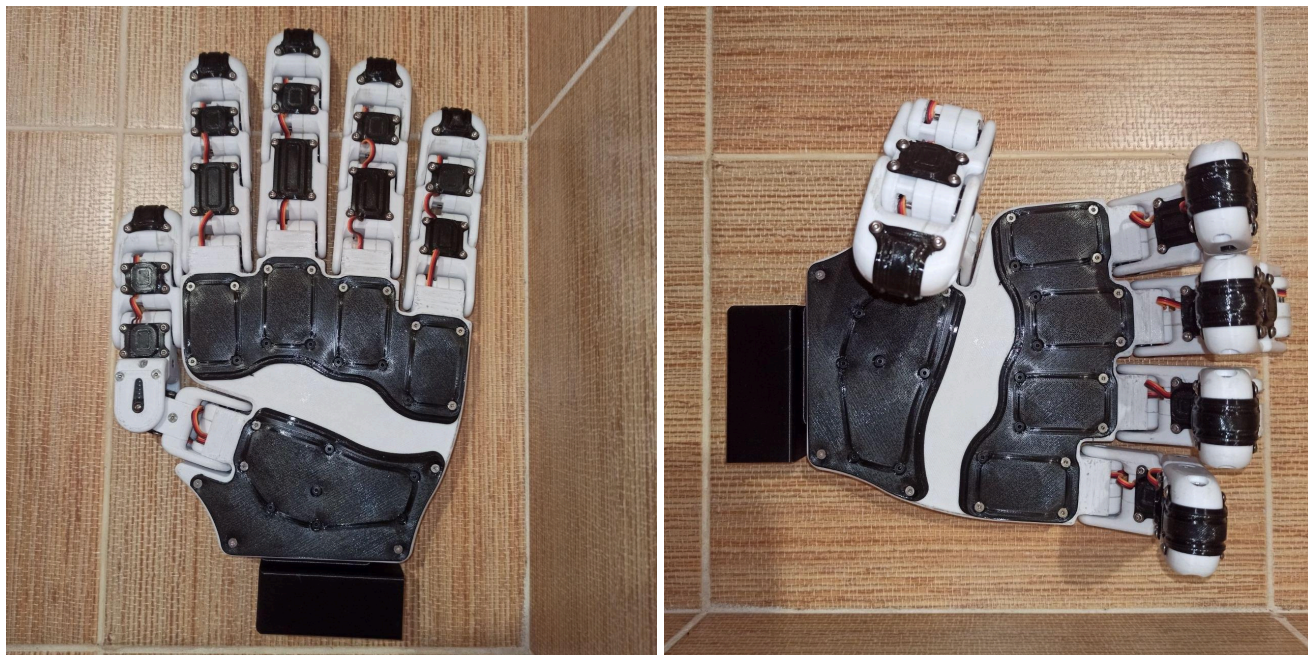


Рисунок 4.2 — Фото готової роботизованої руки

Кожен палець має три рухомі суглоби (базовий, проксимальний та дистальний), які забезпечують природну амплітуду згинання й розгинання, а великий палець додатково оснащений базовим шарніром для динамічної зміни просторової орієнтації та протиставлення іншим пальцям. Загалом антропоморфний маніпулятор має 16 незалежних ступенів свободи, кожен із яких індивідуально керується окремим сервоприводом. Така кінематична надмірність дозволяє механічній системі не просто виконувати найпростіші затискні рухи, а з високою точністю копіювати складну моторику людської кисті, забезпечуючи надійне маніпулювання об'єктами різної геометричної форми.

4.1.2 Вибір сервоприводів

Для керування суглобами маніпулятора використовуються мікросервоприводи типу SG90 та MG90S. Обидва типи є компактними і широко доступними, але відрізняються конструкцією редуктора: SG90 має пластиковий

редуктор, MG90S — металевий, що забезпечує вищу зносостійкість на навантажених суглобах. Додаткові характеристики сервоприводів наведені в таблиці 4.1.

Таблиця 4.1 — Характеристики серводвигунів SG90 та MG90S

Характеристика	Сервопривід SG90	Сервопривід MG90S
Крутний момент (при 4.8 В)	1.2 – 1.6 кг·см	1.8 – 2.2 кг·см
Напруга живлення	4.8 – 6.0 В	4.8 – 6.0 В
Вага	9 г	13.4 г
Тип редуктора (шестерні)	Пластиковий (нейлон)	Металевий (алюміній/латунь)

Керування здійснюється через ШІМ-сигнал у діапазоні від 500 до 2500 мкс при частоті 50 Гц. Кожен сервопривод має індивідуально відкалібровані значення мінімального, середнього і максимального PWM — це компенсує механічні відмінності між окремими екземплярами і забезпечує точне позиціонування в межах робочого діапазону.

Оскільки одночасний рух кількох сервоприводів під навантаженням викликає значні стрибки струму, живлення силової частини виконано окремо від логіки мікроконтролера — від зовнішнього джерела 5В з силою струму не менше 15А.

4.1.3 Вибір мікроконтролера

Для керування сервоприводами обрано мікроконтролерну платформу ESP32S3 у поєднанні з драйвером PCA9685. Останній є 16-канальним ШІМ-контролером із підтримкою шини I2C, що дозволяє керувати всіма 16 виконавчими механізмами, використовуючи лише два сигнальні дроти. Це суттєво спрощує схемотехніку пристрою та повністю знімає апаратні обмеження мікроконтролера на кількість вбудованих PWM-виходів.

Схему електричних з'єднань системи керування маніпулятором наведено на кресленику ІК23.130БАК.006 Е1. На цьому документі детально проілюстровано архітектуру апаратних підключень та сполучення всіх електронних компонентів. Зокрема, відображено підключення мікроконтролера ESP32S3 до 16-канального ШІМ-драйвера PCA9685 по шині I2C, а також розподіл ліній керування для всіх шістнадцяти сервоприводів SG90 та MG90S. Як видно з розробленої схеми, архітектура підключення має чіткий та фізично ізольований розподіл на логічну і силову частини, що є критично важливим для стабільної роботи системи під навантаженням. Логічна взаємодія забезпечується підключенням драйвера PCA9685 до мікроконтролера ESP32S3 через інтерфейс I2C: лінії даних (SDA) та тактування (SCL) з'єднані з відповідними GPIO-пінами мікроконтролера, також об'єднані лінії базового логічного живлення (VCC) та спільної землі (GND).

До керуючих виходів PCA9685 (канали від 0 до 15) підключено 16 мікросервоприводів за допомогою стандартного трипровідного інтерфейсу: керуючий сигнал ШІМ, лінія живлення (V+) та земля (GND). З метою захисту мікроконтролера від електромагнітних завад та просідання напруги під час пікових механічних навантажень, силове живлення сервоприводів розмежоване з живленням керуючої логіки. Струм подається безпосередньо на силовий гвинтовий термінал драйвера PCA9685 від зовнішнього потужного джерела постійного струму (5 В, 50 А). Таке схемотехнічне рішення запобігає непередбачуваним перезавантаженням ESP32S3 і гарантує стабільну роботу системи навіть при одночасному та інтенсивному русі всіх ланок маніпулятора.

ESP32S3 отримує команди від ПК через Serial-інтерфейс на швидкості 115200 бод, парсить їх і передає відповідні PWM-значення на драйвер. Вбудований алгоритм ЕМА-згладжування забезпечує плавність рухів на апаратному рівні незалежно від частоти надходження команд.

4.1.4 Засоби отримання та обробки відеопотоку

Для захоплення відеопотоку використовується стандартна USB веб-камера з роздільною здатністю не менше 720p та частотою кадрів 30 FPS. Вимоги до

					ІК23.130БАК.006 ПЗ	Арк. 37
Зм.	Арк.	№ докум.	Підпис	Дата		

камери є мінімальними — MediaPipe коректно працює на будь-якому сучасному споживчому обладнанні без необхідності у спеціалізованих RGB-D сенсорах. На рисунку 4.3 наведено приклад відеокадру з відображенням руки.

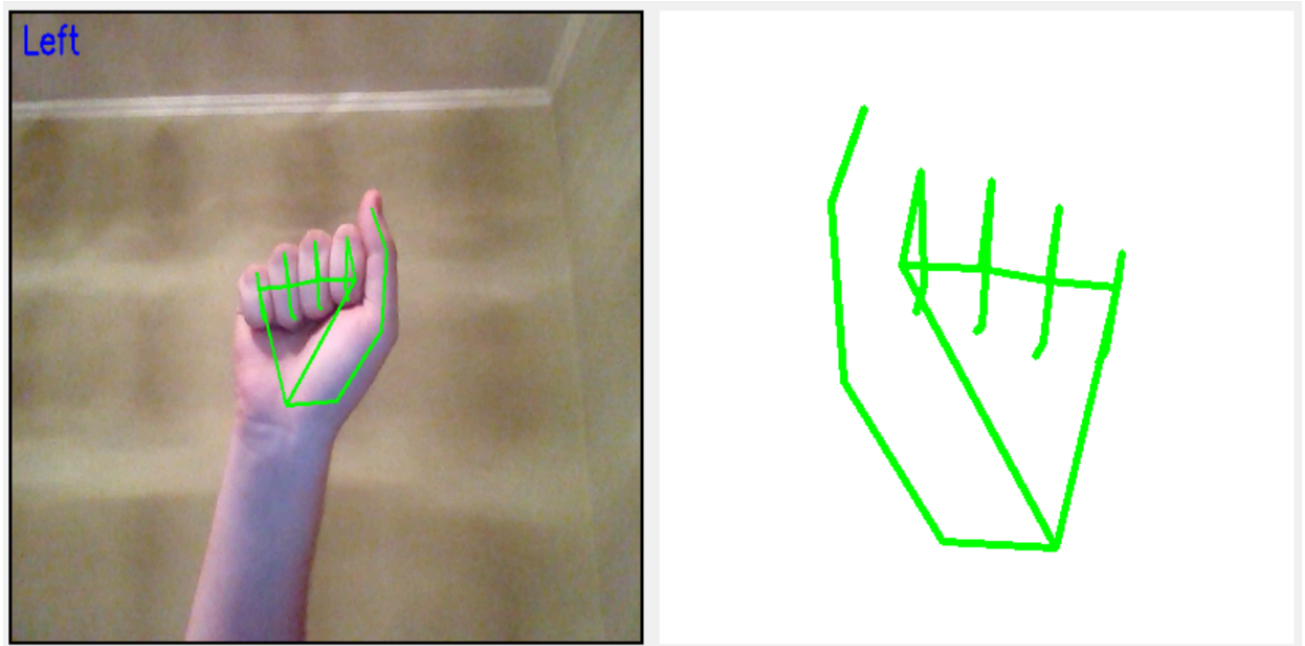


Рисунок 4.3 — Приклад відеокадру з відображенням руки

Відеопотік обробляється бібліотекою OpenCV: кадр віддзеркалюється по горизонталі, конвертується у RGB і передається до MediaPipe для неймережевого інференсу. Весь цикл обробки одного кадру займає менше 40 мс, що забезпечує плавне керування в режимі реального часу.

4.2 Програмне забезпечення для розробки

Програмна частина системи реалізована з використанням кількох інструментів, кожен із яких виконує чітко визначену роль. Вибір кожного з них обґрунтований як технічними вимогами до системи, так і практичними міркуваннями щодо доступності та зручності розробки.

4.2.1 Мова програмування Python

Основна логіка системи — обробка відеопотоку, неймережевий інференс, обчислення кутів суглобів, фільтрація та передача даних — реалізована мовою

Python версії 3.10.9. Вибір обумовлений кількома факторами. По-перше, більшість сучасних бібліотек комп'ютерного зору та машинного навчання мають нативну підтримку Python і найбільш активно розвиваються саме під цю платформу [19]. По-друге, Python дозволяє швидко прототипувати і тестувати алгоритми без надмірних накладних витрат на компіляцію та налагодження.

4.2.2 Середовища розробки

Для написання та налагодження Python-коду використовувався Visual Studio Code із розширеннями для підтримки Python, автодоповнення та інтегрованого термінала [20]. Легкість налаштування і широкі можливості розширення роблять його зручним інструментом для проєктів, що поєднують кілька технологічних стеків.

На етапі дослідження та відпрацювання окремих алгоритмів — зокрема обчислення кутів суглобів і методів фільтрації — використовувалось середовище Jupyter Notebook [21]. Воно дозволяє виконувати код покроково, одразу бачити проміжні результати і візуалізувати дані, що суттєво прискорює процес налагодження математичних перетворень до інтеграції їх у основний проєкт.

Програмування мікроконтролера ESP32S3 виконувалось у середовищі Arduino IDE. Воно забезпечує повний цикл розробки — від написання коду до завантаження прошивки на пристрій — і має вбудований моніторинг Serial-порту, що є зручним при налагодженні обміну даними між ПК і мікроконтролером.

4.2.3 Застосування бібліотек MediaPipe, NumPy та PySerial

Ключову роль у реалізації системи відіграють три зовнішні бібліотеки. MediaPipe від Google забезпечує нейромережеве відстеження 21 ключової точки кисті в режимі реального часу. Використовується модуль HandLandmarker у режимі LIVE_STREAM з асинхронним callback-інтерфейсом. Це дозволяє не блокувати головний цикл обробки під час інференсу.

NumPy використовується для обчислення векторів між точками, скалярних

добуток в і нормалізації значень у допустимі діапазони [22]. Векторизовані операції

ІК23.130БАК.006 ПЗ

Зм.	Арк.	№ докум.	Підпис	Дата
-----	------	----------	--------	------

NumPy швидші за Python-цикли, що критично важливо для обробки даних у реальному часі.

PySerial забезпечує двосторонній обмін даними між ПК і мікроконтролером через Serial-порт. Бібліотека підтримує автоматичне визначення доступних портів, налаштування швидкості передачі та асинхронну роботу з буфером. У системі вона використовується для відправки пакетів керуючих команд на ESP32S3 зі швидкістю 115200 бод.

Для документування програмної взаємодії та розуміння логіки роботи системи розроблено відповідні графічні документи. Блок-схема обміну даними та виклику функцій наведено на кресленику ІК23.130БАК.006 Д1. На ній детально розписано кожен етап життєвого циклу інформації: від захоплення кадру модулем OpenCV, обробки точок нейромережею MediaPipe, розрахунку кутів засобами NumPy на ПК, до пакування команд та їх відправки через PySerial на мікроконтролер ESP32S3, де виконується кінцева генерація ШІМ-сигналів.

Крім того, для ілюстрації ефективності математичних алгоритмів стабілізації, графіки роботи двоступеневих ЕМА-фільтрів наведено на кресленику ІК23.130БАК.006 Д2. На цих графіках наочно показано, як програмний фільтр першого рівня на Python усуває високочастотний міжкадровий шум, а апаратний фільтр другого рівня на ESP32S3 забезпечує плавність ходу сервоприводів. У таблиці 4.2 перелічено усі використані програмні бібліотеки.

Таблиця 4.2 — Перелік використаних програмних бібліотек

Назва бібліотеки	Версія	Призначення у проєкті
MediaPipe	0.10.35	Розпізнавання жестів, детекція та відстеження ключових точок руки (Hand Tracking).
NumPy	2.2.6	Оптимізація обчислювальних процесів, матричні обчислення та фільтрація даних.
PySerial	3.5	Організація обміну даними (Serial) між ПК та мікроконтролером.
OpenCV (cv2)	4.13.0.92	Захоплення відеопотоку з камери та обробка зображень.

Висновки до розділу 4

У четвертому розділі обґрунтовано вибір апаратних та програмних засобів, використаних при розробці інтелектуальної системи керування роботизованим маніпулятором.

Конструкцію маніпулятора спроектовано в Autodesk Inventor 2024 і виготовлено методом FDM 3D-друку з двох матеріалів: PETG для жорстких несучих елементів і TPU для гнучких з'єднувальних тяг. Система має 16 ступенів свободи, кожен із яких керується окремим сервоприводом SG90 або MG90S.

Керування сервоприводами реалізовано через 16-канальний ШІМ-драйвер PCA9685 на базі мікроконтролера ESP32S3. Таке рішення спрощує схему підключення, знімає обмеження на кількість апаратних PWM-виходів і забезпечує стабільне керування всіма каналами одночасно. Живлення силової та логічної частин розділено для запобігання просіданню напруги під час пікових навантажень.

Для захоплення відеопотоку обрано стандартну USB веб-камеру з роздільною здатністю 720p і частотою 30 FPS. Використання звичайного споживчого обладнання без спеціалізованих сенсорів глибини є свідомим рішенням, що знижує вартість системи і розширює коло її потенційного застосування.

Програмна частина реалізована мовою Python 3.10.9 у середовищах Visual Studio Code, Jupyter Notebook і Arduino IDE. Ключові бібліотеки — MediaPipe для

					ІК23.130БАК.006 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		41

нейромережевого трекінгу, NumPy для математичних обчислень і PySerial для обміну даними з мікроконтролером — забезпечують необхідну функціональність при мінімальних апаратних вимогах.

Обраний набір засобів забезпечує оптимальний баланс між доступністю, функціональністю та гнучкістю архітектури, що є важливою умовою для подальшої реалізації та тестування системи.

5 РЕАЛІЗАЦІЯ ІНТЕЛЕКТУАЛЬНОЇ СИСТЕМИ КЕРУВАННЯ

5.1 Реалізація модуля комп'ютерного зору на основі MediaPipe

Модуль комп'ютерного зору є першою і найважливішою ланкою всього конвеєра обробки даних — від його стабільності залежить якість усього, що відбувається далі. Реалізація модуля побудована на бібліотеці MediaPipe у поєднанні з OpenCV для роботи з відеопотоком.

Ініціалізація починається із завантаження моделі HandLandmarker із файлу `hand_landmarker.task`. Модель налаштовується на роботу в режимі `LIVE_STREAM` — це асинхронний режим, у якому результати детекції повертаються через `callback`-функцію, не блокуючи головний цикл програми. Кількість відстежуваних рук обмежена однією — це знижує обчислювальне навантаження і виключає неоднозначність при визначенні активної руки.

Камера ініціалізується через OpenCV за допомогою `cv2.VideoCapture(0)`. На кожній ітерації головного циклу зчитується черговий кадр, який одразу віддзеркалюється по горизонталі для створення ефекту дзеркала. Після цього кадр конвертується з BGR у RGB і передається до MediaPipe через `detect_async` із монотонно зростаючим лічильником часових міток.

Результат інференсу надходить у `callback`-функцію `process_result` і зберігається у змінній `current_result`. На наступній ітерації циклу головний потік зчитує це значення і передає його на подальшу обробку. Така асинхронна схема

					ІК23.130БАК.006 ПЗ	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		42

дозволяє підтримувати стабільну частоту оновлення інтерфейсу незалежно від часу виконання нейромережевого інференсу [23].

Якщо рука в кадрі виявлена — модуль повертає масив із 21 нормалізованої тривимірної координати ключових точок кисті. Якщо рука відсутня — результат порожній, маніпулятор залишається у поточному положенні до появи руки в кадрі. На рисунку 5.1 зображено відеокадри із відображеним скелетом лівої та правої руки та підписом handedness.

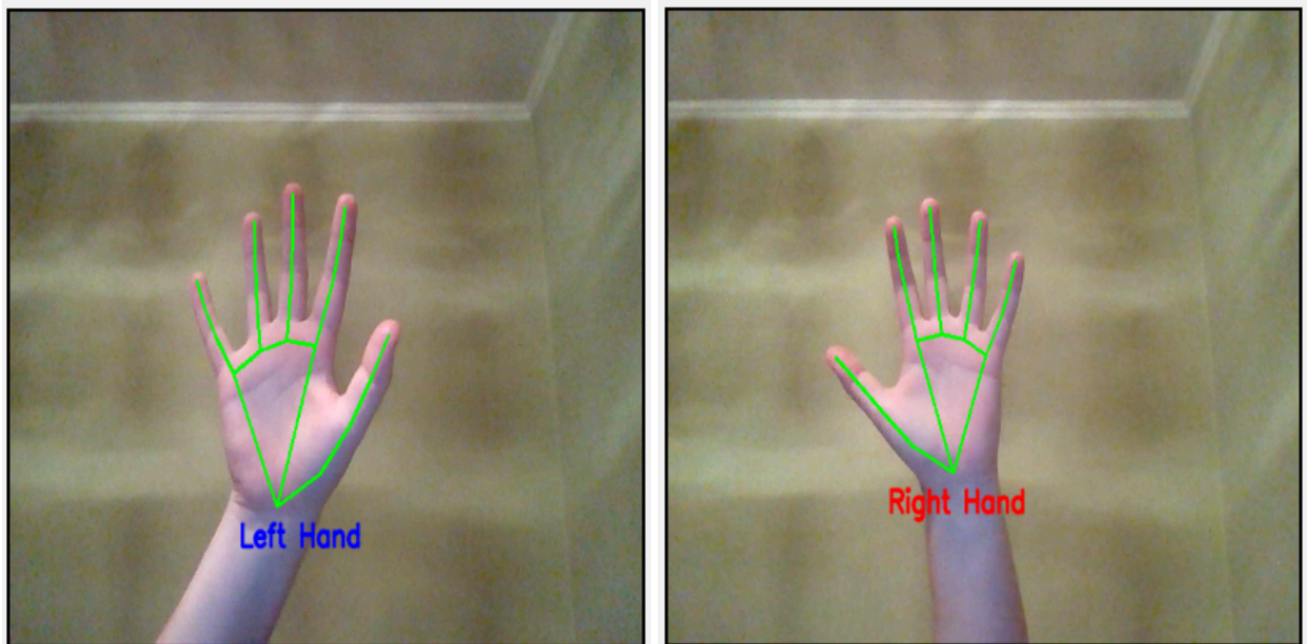


Рисунок 5.1 — Відеокадри із відображенням руки та підписом handedness.

Паралельно з передачею даних до обчислювального ядра модуль відображає скелет руки безпосередньо на відеокадрі — з'єднання між точками малюються через cv2.line, самі точки позначаються колами. Колір підпису руки залежить від її типу: синій для лівої, червоний для правої. Це забезпечує оператору візуальний зворотний зв'язок про те, яку руку система бачить у поточний момент.

5.2 Реалізація алгоритмів обробки та класифікації жестів

Після отримання 21 ключової точки від MediaPipe починається основна обробка даних. Цей етап включає кілька послідовних операцій: визначення типу руки, корекцію орієнтації, згладжування координат і обчислення кутів суглобів.

Оскільки відеокадр віддзеркалюється по горизонталі, мітки MediaPipe інвертуються відносно реальної руки оператора, тому програмно виконується їхня зворотна заміна на коректні значення [24]. Система також визначає, чи повернута рука долонею до камери, чи тильною стороною, шляхом порівняння координат по осі X точки великого пальця (P2) та основи мізинця (P17). Якщо виявлено, що рука перевернута, її координати математично віддзеркалюються відносно центру кисті для збереження правильної логіки роботи сервоприводів.

Далі до розрахованих координат застосовується ЕМА-фільтрація з коефіцієнтом $\alpha=0.5$. При цьому, у разі зміни орієнтації руки або її першій появі в кадрі, буфер попередніх значень автоматично очищається для уникнення артефактів руху та різких стрибків маніпулятора.

Для програмної реалізації математичного апарату (формули 3.4-3.7) було розроблено функцію `get_joint_angle`. Її алгоритм базується на обчисленні скалярного добутку векторів, утворених 3D-координатами трьох суміжних точок суглоба. Для забезпечення обчислень у реальному часі використано векторизовані операції бібліотеки NumPy, що дозволило уникнути повільних ітерацій стандартного Python. Функція автоматично обмежує (`clamp`) вихідний кут у фізично допустимому діапазоні $[90^\circ, 180^\circ]$. Лістинг базової функції розрахунку наведено на рисунку 5.2.

```
def get_joint_angle(p1, p2, p3, clamp=True):
    v1 = np.array(p1) - np.array(p2)
    v2 = np.array(p3) - np.array(p2)
    norm1 = np.linalg.norm(v1)
    norm2 = np.linalg.norm(v2)
    if norm1 == 0 or norm2 == 0:
        return 180.0
    dot = np.dot(v1, v2)
    cos_theta = np.clip(dot / (norm1 * norm2), -1.0, 1.0)
    angle = np.degrees(np.arccos(cos_theta))
    if clamp:
        return np.clip(angle, 90.0, 180.0)
    return angle
```

Рисунок 5.2 — Лістинг функції обчислення кута суглоба

Зазначена функція викликається ітеративно для всіх суглобів пальців кисті. Загалом у процесі обробки одного кадру обчислюється 15 просторових кутів. Разом із базовим каналом, що відповідає за загальну орієнтацію великого пальця, вони формують масив із 16 керуючих значень, які в подальшому перетворюються на команди для сервоприводів. Детальний опис топології зв'язків між ключовими точками та відповідними каналами маніпулятора наведено в таблиці 5.1.

Таблиця 5.1 — Відповідність каналів сервоприводів, точок MediaPipe, суглобів та фаланг пальців

Канал	Палець	Анатомічний суглоб	Точки MediaPipe	PWM мін.	PWM сер.	PWM макс.
0	Мізинець	Дистальний	P20, P19, P18	144	324	500
1	Мізинець	Проксимальний	P19, P18, P17	166	347	500
2	Мізинець	Базовий	P18, P17, P0	177	366	500
3	Безіменний	Дистальний	P16, P15, P14	188	355	500
4	Безіменний	Проксимальний	P15, P14, P13	188	355	366
5	Безіменний	Базовий	P14, P13, P0	211	366	500
6	Великий	Базовий (орієнтація)	P1, P0, P5 / P0, P17	200	353	500
7	Великий	Проксимальний 90°	P2, P1, P0	144	320	500
8	Великий	Проксимальний	P3, P2, P1	200	355	500
9	Великий	Дистальний	P4, P3, P2	177	340	500
10	Середній	Базовий	P10, P9, P0	155	326	500
11	Середній	Дистальний	P12, P11, P10	144	326	500
12	Середній	Проксимальний	P11, P10, P9	144	326	500
13	Вказівний	Дистальний	P8, P7, P6	144	326	500
14	Вказівний	Проксимальний	P7, P6, P5	133	333	500

15	Вказівний	Базовий	P6, P5, P0	144	333	500
----	-----------	---------	------------	-----	-----	-----

5.3. Розробка системи керування роботизованим маніпулятором

Після обчислення кутів суглобів система переходить до етапу формування керуючих сигналів для сервоприводів. Цей процес реалізований частково на стороні програмного забезпечення ПК, де просторові кути перетворюються у цільові команди, і частково на стороні мікроконтролера ESP32S3, де команди перетворюються у фізичний рух за допомогою апаратно генерованих ШІМ-сигналів.

Для адаптації розрахованих кутів під механіку маніпулятора розроблено функцію `calculate_arduino_angle`. Алгоритм перетворення враховує специфіку кожного каналу. Зокрема, канал 6 фіксується у значенні 0° або 180° залежно від типу виявленої руки і відповідає за базову орієнтацію великого пальця. Канал 7 додатково масштабується з коефіцієнтом 3 для компенсації механічних особливостей приводу натягу. Крім того, для лівої руки всі значення інвертуються відносно правої, що забезпечує природність рухів без ускладнення апаратної частини.

Критичною проблемою при зміні руки оператора в кадрі є те, що миттєва зміна орієнтації великого пальця (канал 6) призводить до механічних зіткнень деталей маніпулятора. Для вирішення цієї проблеми програмно реалізовано алгоритм кінцевого автомата (Finite State Machine). Процес переходу поділено на три незалежні фази:

1. примусове складання пальців у безпечне положення (тривалість 2,0 с);
2. перемикання базової орієнтації каналу 6 (загальна тривалість до 3,5 с);
3. завершення переходу та відновлення нормального трекінгу.

Такий підхід повністю виключає ризик пошкодження роботизованої руки. Лістинг програмної реалізації кінцевого автомата наведено на рисунку 5.3.

```

if self.thumb_transition:
    elapsed = current_time - self.thumb_transition_start
    fold_angle = 0 if self.target_hand == "Right" else 180
    old_base = 0 if self.current_hand == "Right" else 180
    new_base = 0 if self.target_hand == "Right" else 180
    if elapsed < 2.0:
        if ch in [7, 8, 9]:
            ard_angle = fold_angle
        elif ch == 6:
            ard_angle = old_base
    elif elapsed < 3.5:
        if ch in [7, 8, 9]:
            ard_angle = fold_angle
        elif ch == 6:
            ard_angle = new_base
    else:
        self.thumb_transition = False
        self.current_hand = self.target_hand
        self.log("Перекидання завершено.")
else:
    if ch == 6:
        ard_angle = 0 if self.current_hand == "Right" else 180

```

Рисунок 5.3 — Лістинг алгоритму кінцевого автомата великого пальця

На стороні мікроконтролера отримані значення кутів перетворюються у ширину імпульсу через двосегментне лінійне відображення з індивідуальними калібрувальними значеннями кожного сервопривода. Після встановлення цільового значення кожні 10 мс запускається цикл ЕМА-згладжування з коефіцієнтом $\alpha = 0.15$. Канал 6 позначений у прошивці як FAST_SERVO і перемикається миттєво без згладжування для швидкого перекидання бази. Решта каналів плавно наближаються до цільового значення, що усуває різкі ривки та захищає механіку від пікових навантажень.

Повний алгоритм роботи системи, що охоплює процеси від захоплення відеокадру та детектування жестів до фізичного руху маніпулятора, проілюстровано на блок-схемі (наведено на кресленику ІК23.130БАК.006 Д1). Цей графічний документ демонструє логіку виконання програми, умови розгалуження, виклики підпрограм та потік перетворення інформації між програмним ядром і апаратною платформою. Загальний опис алгоритму є наступним.

Спочатку, після отримання відеокадру, обчислювальне ядро на базі мови Python (модуль MediaPipe) виконує детекцію кисті та розраховує просторові кути суглобів у діапазоні від 90° до 180°. Одразу після цього отримані значення передаються до функції calculate_esp_angle(), яка відповідає за перетворення цих значень у формат мікроконтролера, адаптуючи їх під механічні особливості та тип

виявленої руки. На діаграмі ці дії винесені в окремі логічні блоки обробки даних на стороні ПК.

Наступним кроком для усунення високочастотного міжкадрового шуму до розрахованих кутів застосовується програмна ЕМА-фільтрація першого рівня [25]. Після згладжування готові кутові значення пакуються у текстові команди встановленого формату та передаються через модуль PySerial. Передача здійснюється через послідовний інтерфейс (Serial) на швидкості 115200 бод, що гарантує своєчасну доставку пакетів даних до апаратної частини без переповнення буфера.

Далі, після надходження інформації на мікроконтролер ESP32S3, відбувається розділення отриманого рядка на окремі значення номера каналу та цільового кута. На основі цих значень система виконує математичне відображення (функція `map()`) кутів у відповідну ширину імпульсу (PWM), враховуючи індивідуальні калібрувальні налаштування кожного окремого сервопривода.

Після цього запускається алгоритм апаратного згладжування. Оскільки фізичний рух маніпулятора має бути плавним і позбавленим різких ривків, до цільових значень PWM застосовується другий рівень ЕМА-фільтрації, що оновлюється кожні 10 мс. Отриманий згладжений ШІМ-сигнал передається на драйвер PCA9685, який безпосередньо генерує імпульси для сервоприводів.

У результаті роботизована рука виконує фізичний рух, синхронно відтворюючи жест оператора, після чого система знов очікує на надходження нових пакетів даних від обчислювального ядра.

5.4. Інтеграція програмних та апаратних компонентів

Програмна і апаратна частини системи працюють як єдиний комплекс завдяки чітко організованому інтерфейсу взаємодії. З боку програмного забезпечення на базі мови Python розроблено графічний інтерфейс користувача (GUI) із використанням бібліотеки Tkinter [26], який об'єднує відеопотік, системну консоль і налаштування підключення в одному вікні. З боку апаратної частини розгорнуто прошивку ESP32S3, що безперервно прослуховує послідовний порт і

реагує на нові команди з мінімальною затримкою.

ІК23.130БАК.006 ПЗ

Арк.

48

Змн.	Арк.	№ докум.	Підпис	Дата

Розроблений графічний інтерфейс візуально та логічно розділено на три функціональні зони. Зліва розташовано область відеокадру, на яку в реальному часі накладається граф (скелет) розпізнаної руки. По центру розміщено окреме віртуальне полотно з 3D-проекцією кисті, побудованою виключно на основі нормалізованих координат, отриманих від моделі MediaPipe. Справа знаходиться панель керування підключенням, що включає випадаючий список доступних СОМ-портів, кнопку ініціалізації з'єднання та повзунок налаштування затримки виводу логів.

Системна консоль, розміщена у нижній частині вікна, виводить повідомлення в реальному часі: статус підключення, помилки передачі даних та інформацію про зміну активної руки. Вона реалізована як текстовий віджет із функцією автопрокрутки та контрастним темним фоном для підвищення читабельності.

Для забезпечення зручності експлуатації, при запуску програма автоматично сканує доступні апаратні порти через бібліотеку `serial.tools.list_ports` і заповнює випадаючий список. Процеси підключення та відключення програмно об'єднані в єдиний логічний блок із захистом від подвійного спрацьовування. При успішному з'єднанні елемент керування змінює колір, що слугує інтуїтивним візуальним індикатором стану. У разі виникнення будь-якої помилки передачі пакетів система автоматично ініціює програмне розірвання з'єднання, щоб уникнути накопичення битих даних у буфері мікроконтролера.

Паралельно з основним відеокадром система відмальовує просторову проекцію скелета руки на білому полотні. Для цього 3D-координати нормалізуються, адаптивно масштабуються під розмір області виводу та відмальовуються за допомогою графічних примітивів бібліотеки OpenCV.

Процес масштабування відбувається динамічно під час обробки кожного кадру: модель кисті завжди залишається відцентрованою і займає близько 70% площі полотна незалежно від фізичної відстані руки оператора до об'єктива камери. Це забезпечує стабільне візуальне відображення просторової моделі.

Особливістю архітектури є вирішення проблеми багатопотоковості. Оскільки графічна бібліотека Tkinter не підтримує пряме оновлення зображень із

					ІК23.130БАК.006 ПЗ	Арк.
						49
Змн.	Арк.	№ докум.	Підпис	Дата		

поток OpenCV в неблокуючому режимі, кожен кадр програмно конвертується у сумісний формат PIL.ImageTk.PhotoImage. Головний цикл оновлення графіки та читання камери запускається через неблокуючий таймер root.after(). Такий підхід дозволяє головному потоку операційної системи паралельно обробляти події інтерфейсу (натискання кнопок, вибір портів) та малювати кадри без ефекту "зависання" вікна.

5.5 Організація обміну даними через PySerial

Обмін даними між Python та мікроконтролером є критичною ланкою комплексу. Будь-яка затримка або помилка передачі впливає на плавність рухів маніпулятора. Тому організація цього каналу за допомогою PySerial потребує суворої регламентації формату даних та частоти їх відправки.

Кожна команда для окремого сервопривода передається у вигляді стандартизованого текстового рядка (див. формулу 5.1):

$$A:\{\text{канал}\}:\{\text{кут}\}\backslash n \quad (5.1)$$

де A — стартовий маркер команди (кутова); {канал} — номер апаратного каналу керування (від 0 до 15); {кут} — цільове значення кута у градусах (ціле число від 0 до 180); \n — символ завершення пакета.

За один ітераційний цикл програма формує суцільний пакет із 16 таких рядків, що відповідають поточним станам усіх приводів. З метою оптимізації навантаження, пакет відправляється не на кожен зчитаний кадр відеопотоку, а з мінімально допустимим інтервалом 20 мс між відправками.

Це забезпечує максимальну частоту команд на рівні близько 50 пакетів на секунду. Такої частоти цілком достатньо для плавного керування механікою, і при цьому вона не перевантажує апаратний буфер UART мікроконтролера ESP32S3. Без програмного обмеження частоти (при 30 FPS відеопотоку) буфер мікроконтролера швидко переповнювався б, що призводило б до втрати команд та ривків маніпулятора.

На стороні мікроконтролера реалізовано алгоритм побайтового зчитування Serial-буфера в кожній ітерації головного циклу. При виявленні стартового маркера A програма послідовно зчитує номер каналу та значення кута за

допомогою вбудованих функцій парсингу числових значень. Після отримання цих ключових параметрів залишок рядка до символу перенесення каретки (`\n`) примусово очищується для запобігання розсинхронізації потоку даних.

Важливим елементом надійності є вбудована апаратна валідація даних: перевірка приналежності зчитаного каналу до діапазону $[0; 15]$ та кута до діапазону $[0; 180]$. Це захищає механіку від виконання некоректних команд, які можуть виникнути у випадку часткової втрати пакету або впливу електромагнітних завад на лінію зв'язку.

На стороні Python будь-яка виняткова ситуація під час відправки перехоплюється блоком обробки виключень (`try-except`). При виникненні апаратної помилки система автоматично викликає функцію розірвання з'єднання, скидає прапорець підключення і виводить відповідне повідомлення в системну консоль. Завдяки цьому оператор одразу бачить проблему (наприклад, випадкове від'єднання кабелю) і може відновити зв'язок одним натисканням кнопки без необхідності повного перезапуску програмного комплексу.

5.6 Оптимізація обчислень із NumPy

Обробка відеопотоку в реальному часі вимагає максимальної оптимізації математичних операцій. При частоті 30 кадрів на секунду (FPS) на обробку одного кадру відводиться не більше 33 мс, значна частина з яких витрачається на обчислення просторових кутів суглобів, фільтрацію та масштабування координат. Саме на цьому етапі використання бібліотеки NumPy дає відчутний виграш у продуктивності.

Замість ітеративної обробки кожної ключової точки за допомогою стандартних циклів Python, усі 21 координати одразу конвертуються у двовимірний масив форми $(21, 3)$ для просторових 3D-даних та $(21, 2)$ для 2D-проекції. Усі подальші перетворення, такі як визначення меж, обчислення геометричного центру та масштабування, виконуються над цілим масивом за допомогою векторних операцій на рівні скомпільованого C-коду.

Зокрема, функція `get_joint_angle` повністю реалізована через NumPy-операції, що дає прискорення у кілька разів. Вбудована функція обмеження `np.clip` виконує подвійну роль: запобігає виходу косинуса кута за межі $[-1; 1]$ через математичні похибки обчислень чисел з рухомою комою і одночасно затискає кінцевий результат у фізично допустимий діапазон без використання додаткових умовних перевірок.

Згладжування 3D-координат також реалізовано як єдина векторна операція над масивами. За один прохід обчислювача одночасно згладжуються всі 63 числові значення ($21 \text{ точка} \times 3 \text{ координати}$) без жодної ітерації по точках.

Під час побудови 3D-проекції для графічного полотна масштаб і зміщення обчислюються векторно: функції пошуку мінімуму та максимуму (`np.min` та `np.max`) з параметром `axis=0` одночасно знаходять екстремуми по обох координатах за один прохід масиву.

Загалом, використання бібліотеки NumPy дозволяє виконувати всі математичні перетворення над даними руки значно швидше порівняно зі звичайними циклами, що є ключовою умовою стабільної роботи системи в режимі реального часу [27].

Висновки до розділу 5

У п'ятому розділі розроблено та програмно реалізовано комплекс інтелектуальної системи жестового керування маніпулятором. Модуль комп'ютерного зору на базі MediaPipe HandLandmarker працює в режимі асинхронного потоку. Відеокадри зчитуються через OpenCV, віддзеркалюються та конвертуються у потрібний формат перед передачею до нейромережі.

Алгоритм обробки жестів включає визначення типу руки з інверсією міток, корекцію орієнтації долоні, просторове ЕМА-згладжування координат та обчислення 15 кутів суглобів через скалярний добуток. Розраховані кути перетворюються у команди з урахуванням механічних особливостей окремих каналів. Для безпечної зміни орієнтації великого пальця реалізовано триетапний алгоритм кінцевого автомата. На стороні ESP32S3 виконується лінійне

відображення кутів у ширину імпульсу та апаратна фільтрація сигналу з частотою 100 Гц.

Інтеграцію програмних і апаратних компонентів забезпечено через графічний інтерфейс із відображенням відеопотоку та 3D-моделі руки, а також канал UART з обмеженням (до 50 пакетів/с) для запобігання переповненню буфера. Оптимізація ядра векторними операціями NumPy гарантує швидкодію обробки в реальному часі.

Отже, створено працездатну систему жестового керування. Оптимізовані математичні перетворення та дворівневе згладжування забезпечили плавність рухів маніпулятора й захист від колізій. Реалізований комплекс відповідає вимогам і переходить до тестування.

6 ТЕСТУВАННЯ ІНТЕЛЕКТУАЛЬНОЇ СИСТЕМИ КЕРУВАННЯ

6.1 Методика тестування

Тестування системи проводилось з метою оцінки її працездатності в умовах, наближених до реального використання. На відміну від лабораторних тестів із контрольованим середовищем, перевірка виконувалась при звичайному кімнатному освітленні, на стандартному споживчому обладнанні і без попереднього калібрування камери під конкретного оператора.

Для отримання об'єктивних результатів тестування розбито на чотири незалежних напрями. Перший — перевірка точності розпізнавання ключових точок руки і обчислення кутів суглобів. Другий — вимірювання затримки між рухом руки оператора і реакцією маніпулятора. Третій — оцінка стабільності роботи при зміні умов середовища. Четвертий — виявлення граничних ситуацій і обмежень системи.

Апаратна конфігурація тестового стенду включала персональний комп'ютер із процесором Intel Core i5 11400H та 16 ГБ оперативної пам'яті, стандартну USB веб-камеру з роздільною здатністю 720p і частотою 30 FPS, мікроконтролер

ESP32S3 з драйвером PCA9685 та роботизовану руку з 16 сервоприводами. На рисунку 6.1 зображено тестовий стенд.

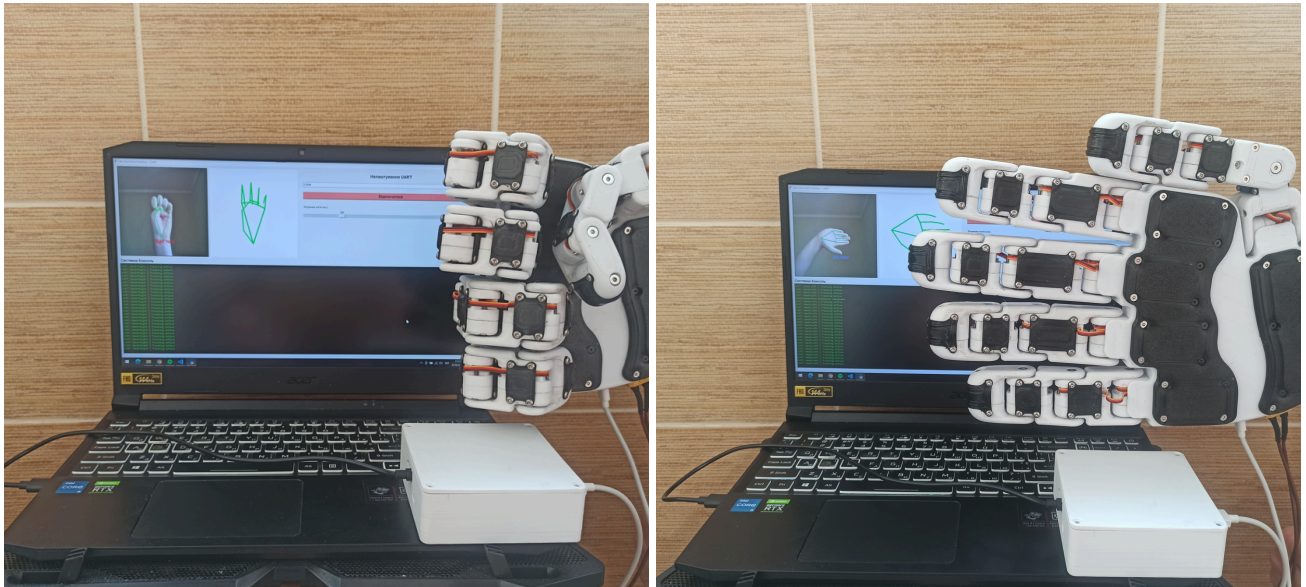


Рисунок 6.1 — Фотографії тестового стенду

Програмне середовище: Python 3.10.9, MediaPipe 0.10, OpenCV 4.8, PySerial 3.5. Операційна система — Windows 10.

Кожен тест повторювався щонайменше 10 разів для різних операторів і різних положень руки. Результати фіксувались через системну консоль програми та зовнішній секундомір для вимірювання затримок. Для оцінки стабільності кутових значень використовувались дані, збережені з Serial-монітора Arduino IDE під час тривалих сесій роботи. У таблиці 6.1 наведено параметри тестового середовища, а саме: апаратна конфігурація, версії ПЗ, умови освітлення.

Таблиця 6.1 — Параметри тестового середовища

Категорія	Параметр	Значення
Апаратна конфігурація	Персональний комп'ютер	Процесор Intel Core i5 11400H, 16 ГБ оперативної пам'яті
	Камера	Стандартна USB веб-камера, роздільна здатність 720p, частота 30 FPS
	Мікроконтролер	ESP32S3 з драйвером PCA9685

	Виконавчий механізм	Роботизована рука з 16 сервоприводами
Програмне забезпечення	Операційна система	Windows 10
	Мова програмування	Python 3.10.9
	Бібліотеки	MediaPipe 0.10, OpenCV 4.8, PySerial 3.5
Умови освітлення	Сценарії тестування	Яскраве денне світло від вікна, стандартне кімнатне освітлення, слабе освітлення настільною лампою, контрове освітлення

6.2 Оцінка точності розпізнавання жестів

Основне завдання цього етапу — перевірити, наскільки коректно система визначає положення 21 ключової точки кисті і наскільки стабільними є обчислені кути суглобів у статичному і динамічному режимах.

Для оцінки стабільності в статичному режимі оператор утримував руку нерухомо протягом 10 секунд у кількох фіксованих положеннях — повністю розкрита долоня та стиснутий кулак різними руками. Протягом цього часу через Serial-монітор фіксувались значення кутів для всіх 16 каналів.

Без фільтрації значення кутів коливались у межах $\pm 8\text{--}12^\circ$ від середнього — це прямий наслідок шуму камери і дискретності нейромережевого детектора. Після застосування ЕМА-фільтра з $\alpha=0.5$ на стороні Python розкид скоротився до $\pm 2\text{--}3^\circ$. Апаратний ЕМА на ESP32S3 з $\alpha=0.15$ додатково згладив PWM-сигнал, що повністю усунуло видиме тремтіння сервоприводів у статичному положенні. В таблиці 6.2 наведено порівняння кутових значень до і після фільтрації для ключових каналів.

Таблиця 6.2 — Порівняння кутових значень до і після фільтрації.

Номер каналу	Палець та анатомічний суглоб	Значення до фільтрації	Значення Python EMA ($\alpha=0.5$)	Стан після ESP32S3 EMA ($\alpha=0.15$)
0	Мізинець (дистальний)	± 9	± 2	Стабільний, тремтіння відсутнє
4	Безіменний (проксимальний)	± 8	± 2	Стабільний, тремтіння відсутнє
8	Великий (проксимальний)	± 12	± 3	Мінімальні коливання під навантаженням
12	Середній (проксимальний)	± 9	± 2	Стабільний, тремтіння відсутнє
13	Вказівний (дистальний)	± 10	± 2	Стабільний, тремтіння відсутнє

При плавних рухах пальців система коректно відстежувала зміни кутів у всьому робочому діапазоні. Маніпулятор впевнено повторював такі конфігурації: повне розкриття і стискання кисті, ізольований рух вказівного пальця, послідовне згинання пальців і базові жести з різними комбінаціями.

Найбільша похибка зафіксована на великому пальці через його складну кінематику. Під час різких рухів (канали 7–9) виникали короткочасні стрибки, які ЕМА-фільтр не встигав згладити, спричиняючи помітну затримку. Динаміку зміни кута 13 при плавному та різкому згинанні наведено на рисунку 6.2 та рисунку 6.3 відповідно.

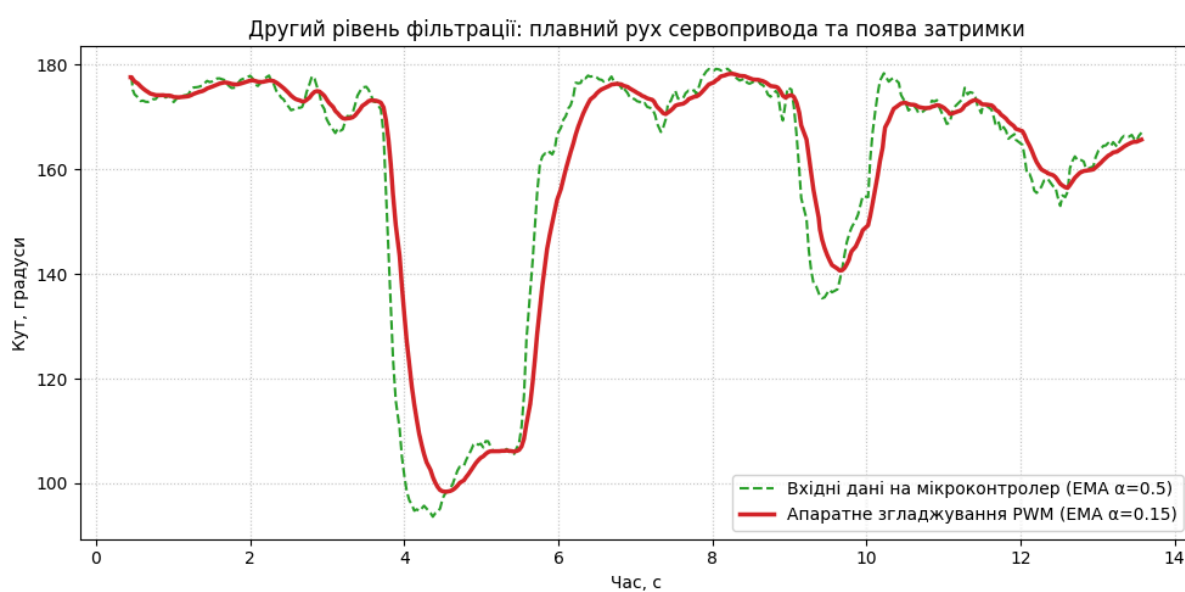


Рисунок 6.2 — Графіки зміни кута каналу 13 (дистальна фаланга вказівного пальця) при плавному згинанні

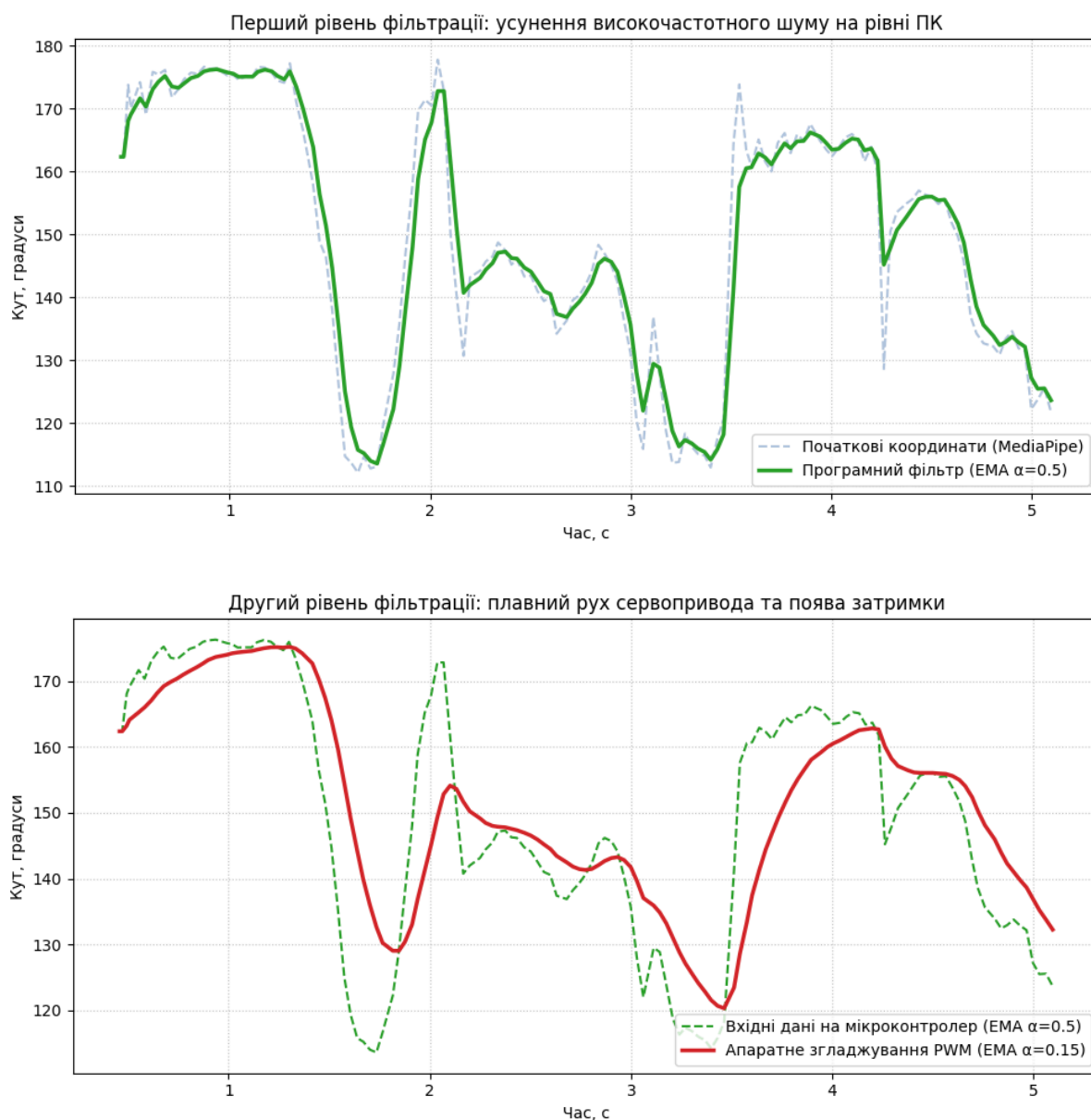


Рисунок 6.3 — Графіки зміни кута каналу 13 (дистальна фаланга вказівного пальця) при різкому згинанні

У контрольованих умовах (рівномірне освітлення від 300 люкс, стабільне фронтальне положення на відстані 50-80 см) система продемонструвала точність визначення типу руки на рівні 98-99%. Зниження точності спостерігалось лише у граничних випадках (різке перекриття кисті, контрове світло або вихід за межі робочої дистанції). При повороті кисті тильною стороною до камери алгоритм інверсії координат спрацьовував стабільно, проте в момент проходження кута 90° (ребро долоні) детектор міг короткочасно втрачати об'єкт, що вимагало утримання поточного стану сервоприводів до відновлення трекінгу.

6.3 Аналіз часу відгуку та впливу затримок системи

Затримка між рухом руки оператора і відповідним рухом маніпулятора — один із ключових показників якості системи керування в реальному часі. Навіть функціонально коректна система стає незручною у використанні, якщо ця затримка перевищує поріг сприйняття людини близько 100–150 мс.

Повна затримка системи складається з кількох послідовних етапів. Кожен із них вносить свій внесок у загальний час відгуку. Складові системи та їх орієнтовані значення наведено в таблиці 6.3.

Таблиця 6.3 — Складові затримки системи та їх орієнтовні значення

Етап	Опис	Орієнтовний час
Захоплення кадру	OpenCV зчитує кадр із камери	~5 мс
Нейромережевий інференс	MediaPipe визначає ключові точки	~15–20 мс
Обчислення кутів	NumPy, ЕМА-фільтр, маппінг	~2–3 мс
Серіалізація та передача	PySerial, Serial 115200 бод	~3–5 мс
Апаратний ЕМА	Плавне наближення до цільового PWM	~20–40 мс
Механічна інерція	Фізичний рух сервопривода	~10–20 мс
Загалом		~55–90 мс

Виміряна затримка від помітного руху руки до початку руху маніпулятора склала в середньому 60–80 мс при нормальному навантаженні системи. Це значення знаходиться нижче порогу сприйняття людини, що забезпечує відчуття природної і безперервної взаємодії.

Апаратний ЕМА із $\alpha=0.15$ вносить найбільший внесок у затримку серед програмних компонентів. Це свідомо інженерна компромісна межа: менший коефіцієнт дає плавніший рух, але повільніший відгук. При збільшенні α до 0.3–0.4 затримка скорочується до 30–50 мс, але на повільних рухах з'являється

помітне тремтіння сервоприводів. Значення 0.15 виявилось оптимальним для даної механічної конструкції. На рисунку 6.4 зображено графік порівняння відгуку руки при різних значеннях α .



Рисунок 6.4 — Графік порівняння відгуку при різних значеннях α

При роботі системи без CNN-модуля завантаження процесора становило 18–25%. Це залишає достатній запас для фонових процесів операційної системи і не призводить до пропуску кадрів. Частота обробки стабільно тримається на рівні 28–30 ітерацій циклу за секунду, що відповідає частоті вхідного відеопотоку.

Обмеження у 20 мс між пакетами дало частоту відправки близько 45–50 пакетів на секунду. Буфер ESP32S3 при цьому не переповнювався — команди надходили і оброблялись рівномірно без накопичення черги. При знятті обмеження і відправці команд на кожен кадр буфер заповнювався протягом 3–5 секунд, після чого маніпулятор починав рухатись із характерними ривками і затримками.

6.4 Аналіз стабільності роботи та впливу шумів

Реальні умови використання системи рідко бувають ідеальними — освітлення змінюється, рука знаходиться на різних відстанях від камери,

оператори рухаються з різною швидкістю. Цей етап тестування перевіряв, наскільки система зберігає працездатність при відхиленні від оптимальних умов.

Тестування системи проводилось у чотирьох характерних сценаріях освітлення: яскраве денне світло від вікна, стандартне кімнатне освітлення, слабе освітлення за допомогою настільної лампи, а також контрове освітлення, коли джерело світла розташоване позаду оператора. Такі умови обрано для перевірки стійкості алгоритму комп'ютерного зору до різних рівнів інтенсивності та напрямку освітлення.

У кожному сценарії оцінювалась стабільність детекції руки, точність визначення ключових точок кисті та частота втрати трекінгу. Отримані результати дозволили визначити межі коректної роботи системи та вплив умов освітлення на якість розпізнавання.

У таблиці 6.4 наведено узагальнені результати тестування стабільності детекції руки при різних умовах освітлення.

Таблиця 6.4 — Стабільність детекції руки при різних умовах освітлення

Умови освітлення	Стабільність детекції	Якість кутів
Яскраве денне	Висока	Висока
Стандартне кімнатне	Висока	Висока
Слабке (~50 лк)	Середня	Знижена
Контрове	Низька	Незадовільна

При стандартному кімнатному освітленні система працювала стабільно протягом усіх тестових сесій без жодних втрат руки в кадрі. При слабкому освітленні камера збільшувала витримку, через що рухи руки розмивались — MediaPipe іноді втрачав окремі пальці на кілька кадрів поспіль, що призводило до коротких стрибків у керуючих сигналах. Контрове освітлення виявилось найбільш проблематичним: рука перетворювалась на темний силует і система стабільно визначала лише зап'ястя і основи пальців.

Тестувались три робочі дистанції: близька (30–40 см), оптимальна (50–80 см) і далека (90–120 см). На рисунку 6.5 наведено порівняння якості скелету руки на різних відстанях.

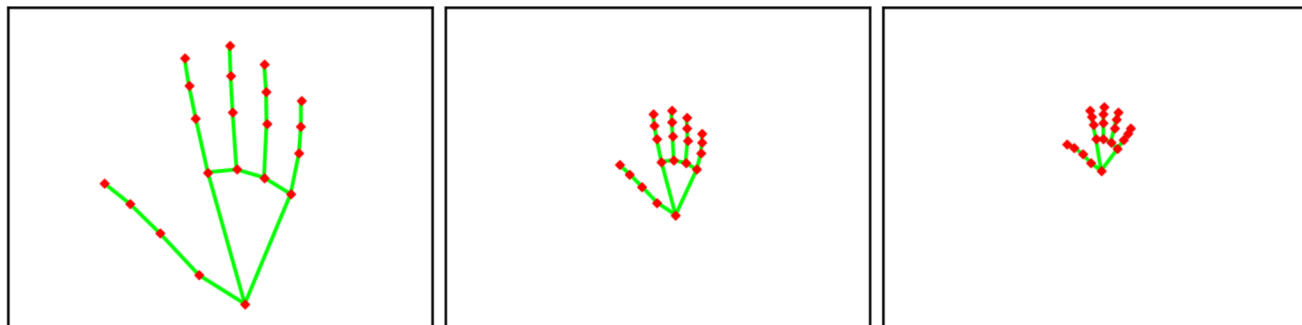


Рисунок 6.5 — Порівняння якості скелету руки на різних дистанціях:
близька, оптимальна та далека відстані

На дуже близькій відстані пальці могли частково виходити за межі кадру. Під час широких рухів кисті система продовжувала працювати, але точність визначення крайніх точок знижувалась. На оптимальній та близькій відстані якість детекції була найвищою. На далекій відстані на кисть припадало менше пікселів, і під час дрібних рухів пальців кути змінювались менш чутливо — маніпулятор реагував з відчутним запізненням на незначні згини.

Під час повільних і середніх рухів система працювала без видимих артефактів. При різких швидких рухах, наприклад, швидкому стисканні кулака — ЕМА-фільтр не встигав повністю відпрацювати перехід, і маніпулятор досягав цільового положення на 200–300 мс пізніше за реальний рух руки. Для плавного керування це несуттєво, але при швидких командних жестах затримка помітна.

Система тестувалась у безперервному режимі протягом 30 хвилин. Витоків пам'яті та деградації продуктивності не виявлено. Частота обробки кадрів залишалась стабільною. Температура сервоприводів зростала, але залишалась у допустимих межах.

Механізм плавного переходу при зміні руки відпрацьовував коректно в усіх випадках. Час переходу — близько 3.5 секунд — достатній для безпечного перекладання великого пальця. У разі повторної зміни руки під час активної фази система завершувала поточний перехід перед запуском нового.

6.5 Узагальнення отриманих результатів

Проведене тестування підтвердило загальну працездатність системи і дозволило кількісно оцінити її ключові характеристики. Разом із тим виявлено ряд обмежень, що визначають межі практичного застосування.

Система демонструє стабільну роботу в режимі реального часу при стандартних умовах використання. Основні виміряні показники наведено в таблиці 6.5.

Таблиця 6.5 — Підсумкові характеристики системи

Показник	Значення
Затримка відгуку	60–80 мс
Частота обробки кадрів	30–35 FPS
Стабільна частота відправки команд	45–50 пакетів/с
Розкид кутів після фільтрації	$\pm 2-3^\circ$
Завантаження CPU	18–25%
Час безперервної роботи	30+ хвилин
Стабільність детекції (норм. освітлення)	Висока

Двоступенева фільтрація повністю виконала своє завдання: тремтіння маніпулятора в статичних положеннях усунуто, плавність рухів під час динамічного керування є прийнятною для практичного використання. Розподіл обчислень між Python і ESP32S3 виправдав себе — жоден із рівнів не став вузьким місцем при нормальному навантаженні.

Окремої уваги в процесі подальшого розвитку системи потребують наявні обмеження її функціонування.

Залежність від умов освітлення залишається найсуттєвішим практичним обмеженням. Контрове і слабе освітлення суттєво знижують якість детекції, що унеможливорює надійне використання системи поза контрольованим середовищем без додаткового налаштування.

Великий палець є найменш стабільним елементом системи. Його нестандартна кінематика і широкий діапазон рухів призводять до більшого розкиду кутових значень порівняно з іншими пальцями. Механізм переходу між руками займає 3,5 секунди. При частій зміні рук це може відчуватись як затримка.

При різких рухах ЕМА-фільтр вносить додаткову затримку відгуку. Підвищення коефіцієнта згладжування для прискорення реакції призводить до тремтіння в статиці. Цей компроміс не вдається вирішити в межах простої однорівневої фільтрації.

Нарешті, система розрахована на одну руку в кадрі. В процесі появи другої руки або сторонніх об'єктів, схожих на кисть, MediaPipe може хибно визначити активну руку, а захист від таких ситуацій у поточній версії мінімальний.

Для вдосконалення системи варто застосувати адаптивний коефіцієнт ЕМА (для балансу плавності й відгуку) та попередню обробку зображень (для стійкої детекції). Додатково надійність і зручність підвищать фільтрація хибних спрацювань на сторонні об'єкти та оптимізація механіки великого пальця для швидкої зміни рук.

Висновки до розділу 6

У шостому розділі було виконано комплексне тестування розробленої системи жестового керування роботизованим маніпулятором, спрямоване на верифікацію ключових показників роботи системи за чотирма основними напрямками: точність розпізнавання, швидкодія, стабільність функціонування в різних умовах та аналіз технічних обмежень.

Аналіз точності розпізнавання підтвердив високу ефективність впровадженої двоступеневої системи фільтрації даних. Завдяки застосуванню алгоритму ЕМА вдалося досягти суттєвого покращення параметрів руху: розкид кутових значень було скорочено з початкових $\pm 8-12^\circ$ до $\pm 2-3^\circ$. Це рішення дозволило повністю нівелювати ефект видимого тремтіння маніпулятора під час знаходження в статичних положеннях. Система коректно ідентифікує тип руки та орієнтацію долоні в усіх стандартних положеннях.

Оцінка швидкодії продемонструвала належну оптимізацію програмного забезпечення. Виміряна затримка відгуку системи складає 60–80 мс, що знаходиться нижче порогу сприйняття людини та забезпечує необхідну природність взаємодії з маніпулятором. Стабільність частоти обробки кадрів на рівні 30–35 FPS при помірному завантаженні центрального процесора (18–25%) є підтвердженням правильності обраної архітектури та ефективності використання векторизованих обчислень бібліотеки NumPy.

Тестування стабільності в реальних умовах підтвердило надійність системи при стандартному кімнатному освітленні та використанні оптимальної дистанції від 50 до 80 см. Тривалі випробування протягом 30 хвилин довели відсутність деградації продуктивності, проте виявили низку факторів, що обмежують можливості системи. Зокрема, встановлено чутливість до контрового та слабкого освітлення, зниження точності на граничних дистанціях, недостатню стабільність відстеження великого пальця, фіксовану тривалість механізму переходу між руками та відсутність алгоритмів захисту від хибних детекцій. Систематизація цих обмежень дозволила окреслити конкретні напрями подальшого вдосконалення системи, які визначають пріоритети її розвитку в майбутньому.

ВИСНОВКИ

У результаті виконання дипломного проекту було успішно досягнуто поставленої мети та вирішено всі науково-практичні завдання щодо створення інтелектуальної системи безконтактного керування антропоморфним роботизованим маніпулятором у режимі реального часу. Підсумовуючи виконану роботу, можна зробити такі висновки щодо кожного етапу розробки:

1. На етапі аналізу предметної області досліджено сфери застосування антропоморфних систем. Встановлено, що класичні методи керування є надто складними або обмежують рухливість оператора, що повністю обґрунтовує потребу у створенні інтуїтивних безконтактних інтерфейсів на основі комп'ютерного зору.

2. Аналіз існуючих рішень підтвердив, що використання контактних маркерних систем чи рукавичок є ергономічно неефективним, а класичний комп'ютерний зір є вразливим до змін фону. Тому для реалізації обрано підхід на основі нейромережевого трекінгу (MediaPipe) у поєднанні з відкритою апаратною платформою ESP32S3, що забезпечило оптимальний баланс точності та апаратної доступності.

3. Під час проєктування розроблено архітектуру інформаційного конвеєра «зір — інтерпретація — керування». Сформовано математичний апарат розрахунку просторових кутів суглобів, створено метод інверсії координат для динамічного розпізнавання лівої/правої руки та обґрунтовано необхідність двоступеневої ЕМА-фільтрації для усунення апаратних шумів.

4. Обґрунтовано та інтегровано апаратно-програмну базу комплексу. Механічну частину реалізовано на базі надрукованої на 3D-принтері кисті з 16 ступенями свободи. Електронна частина базується на мікроконтролері ESP32S3 та ШІМ-драйвері PCA9685, а програмне ядро — на мові Python із залученням бібліотек OpenCV, MediaPipe та NumPy.

5. Програмно-апаратна реалізація успішно об'єднала всі модулі в єдиний комплекс. Оптимізація обчислень векторними операціями NumPy забезпечила стабільну роботу в реальному часі. Розроблено механізм безпечної зміни орієнтації бази великого пальця (кінцевий автомат) та організовано надійний

					ІК23.130БАК.006 ПЗ	Арк.
Зм.	Арк.	№ докум.	Підпис	Дата		66

обмін даними через UART із програмним обмеженням частоти до 50 пакетів на секунду.

6. За результатами комплексного тестування підтверджено ефективність застосованих рішень. Двоступенева фільтрація дозволила повністю усунути тремтіння сервоприводів у статистиці (розкид кутів зменшився з $\pm 8-12^\circ$ до $\pm 2-3^\circ$). Система стабільно працює з частотою 30-35 кадрів на секунду, а загальна затримка відгуку становить 60-80 мс, що знаходиться нижче порогу сприйняття людини і забезпечує ефект безперервної «тіні».

Загалом розроблений програмно-апаратний комплекс продемонстрував високу точність розпізнавання на рівні 98-99% у контрольованих умовах. Встановлено, що система гарантує заявлену стабільність керування виключно за умов достатнього освітлення (від 300 люкс) та збереження робочої дистанції 50-80 см. При порушенні цих параметрів ефективність трекінгу очікувано знижується. Для підвищення загальної надійності в умовах реальної експлуатації у програмний комплекс інтегровано механізм безпечного паркування (Safe Shutdown), який автоматично переводить сервоприводи у нейтральне положення при закритті системи або непередбачуваному розриві зв'язку. Отримані результати підтверджують практичну цінність роботи: створений прототип може слугувати надійною базою для розробки персональних роботів-асистентів або дослідницьких лабораторних стендів.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Bowman M., Zhang J., Zhang X. Intent-based Task-Oriented Shared Control for Intuitive Telemanipulation. Journal of Intelligent & Robotic Systems. 2024. Vol. 110. Art. 167. URL: <https://link.springer.com/article/10.1007/s10846-024-02185-1> (дата звернення: 20.04.2026).
2. Safavi F., Olikkal P., Pei D., Kamal S., Meyerson H., Penumalee V. Emerging Frontiers in Human-Robot Interaction. Journal of Intelligent & Robotic Systems. 2024. Vol. 110. Art. 45. URL: <https://link.springer.com/article/10.1007/s10846-024-02074-7> (дата звернення: 20.04.2026).
3. Манойленко О. П., Калініна К. Р., Лебеденко Ю. О., Дидик М. А. Інтелектуальні системи керування мехатронними системами типу «Cobot» з використанням технологій машинного навчання // Технології та інжиніринг. 2023. №6. С. 35–46. URL: <https://jrnl.knutd.edu.ua/index.php/techeng/article/view/1426/1332> (дата звернення: 21.04.2026).
4. Литвин О., Паньков С. Роботизовані маніпулятори особливого призначення// Технічні науки та технології. 2020. № 1(19). С. 81–88. URL: <https://www.researchgate.net/publication/343340241> (дата звернення: 21.04.2026).
5. World Robotics 2025 – Industrial Robots / International Federation of Robotics. Frankfurt : IFR, 2025. URL: [IFR World Robotics Report](#) (дата звернення: 21.04.2026).
6. Rodinò S., Chiodo M., Corigliano A. та ін. Robotic systems for upper-limb rehabilitation: a systematic review of actuation technologies, kinematic architectures, and clinical translation barriers. Research on Biomedical Engineering. 2026. Vol. 42. Art. 22. URL: <https://link.springer.com/article/10.1007/s42600-026-00469-z> (дата звернення: 21.04.2026).
7. Корсак Р. Роботизація як інноваційний інструмент розвитку міжнародної готельної та туристичної індустрії // Development Service Industry Management. 2025. URL: <https://dsim.khmnu.edu.ua/index.php/dsim/article/view/414> (дата звернення: 21.04.2026).

8. Droukas L., Doulgeri Z., Tsakiridis N. L. et al. A survey of robotic harvesting systems and enabling technologies. Journal of Intelligent and Robotic Systems. 2023. Vol. 107. Article 21. URL: <https://link.springer.com/article/10.1007/s10846-022-01793-z> (дата звернення: 21.04.2026).

9. Wang Y., Hao T., Liu Y., Xiao H., Liu S., Zhu H. Anthropomorphic Soft Hand: Dexterity, Sensing, and Machine Learning // Actuators. 2024. Vol. 13, No. 3. Art. 84. URL: <https://www.mdpi.com/2076-0825/13/3/84> (дата звернення: 22.04.2026).

10. Hussain S., Saeed K., Baimagambetov A. et al. Advancements in Gesture Recognition Techniques and Machine Learning for Enhanced Human–Robot Interaction: A Comprehensive Review. 2024. URL: <https://arxiv.org/abs/2409.06503> (дата звернення: 22.04.2026).

11. Lan G., Wu Y., Hu F., Hao Q. Vision-Based Human Pose Estimation via Deep Learning: A Survey // IEEE Transactions on Human-Machine Systems. 2023. Vol. 53, No. 1. P. 253–268. URL: <https://arxiv.org/pdf/2308.13872> (дата звернення: 22.04.2026).

12. Chen C., Lu C. X., Wang B., Trigoni N., Markham A. DynaNet: Neural Kalman Dynamical Model for Motion Estimation and Prediction // IEEE Transactions on Neural Networks and Learning Systems. 2021. Vol. 32, No. 12. P. 5479–5491. URL: https://www.pure.ed.ac.uk/ws/files/284785241/DynaNet_CHEN_DOA09092021_AFV.pdf (дата звернення: 23.04.2026).

13. Gu F., Zhou Y., Wang Z., Jiang S., He B. A Survey on Robotic Manipulation of Deformable Objects: Recent Advances, Open Challenges and New Frontiers // arXiv. 2023. URL: <https://arxiv.org/pdf/2203.08903> (дата звернення: 24.04.2026).

14. Huang X., Xue Y., Ren S., Wang F. Sensor-Based Wearable Systems for Monitoring Human Motion and Posture: A Review // Sensors. 2023. Vol. 23, No. 22. P. 9047. URL: <https://doi.org/10.3390/s23229047> (дата звернення: 24.04.2026).

15. Ohkawa T., Furuta R., Sato Y. Efficient Annotation and Learning for 3D Hand Pose Estimation: A Survey // International Journal of Computer Vision. 2023. Vol. 131. P. 3193–3206. URL: <https://link.springer.com/article/10.1007/s11263-023-01856-0> (дата звернення: 24.04.2026).

16. Prabhavalkar R., Hori T., Sainath T. N., Schlüter R., Watanabe S. End-to-End Speech Recognition: A Survey // IEEE/ACM Transactions on Audio, Speech, and Language Processing. 2024. Vol. 32. P. 325–351. URL: <https://dl.acm.org/doi/10.1109/TASLP.2023.3328283> (дата звернення: 24.04.2026).
17. Zhang F., Bazarevsky V., Vakunov A., Tkachenka A., Sung G., Chang C. L., Grundmann M. MediaPipe Hands: On-device Real-time Hand Tracking // arXiv. 2020. URL: <https://arxiv.org/abs/2006.10214> (дата звернення: 25.04.2026).
18. Wu H. M., Wei S. H. Skeletal Image Features Based Collaborative Teleoperation Control of the Double Robotic Manipulators // Electronics. 2025. Vol. 14, No. 19. Art. 3897. URL: <https://doi.org/10.3390/electronics14193897> (дата звернення: 25.04.2026).
19. Raschka S., Patterson J., Nolet C. Machine Learning in Python: Main Developments and Technology Trends in Data Science, Machine Learning, and Artificial Intelligence // Information. 2020. Vol. 11, No. 4. Art. 193. URL: <https://doi.org/10.3390/info11040193> (дата звернення: 01.05.2026).
20. Tan J., Chen Y., Jiao S. Visual Studio Code in Introductory Computer Science Course: An Experience Report // arXiv. 2023. URL: <https://arxiv.org/abs/2303.10174> (дата звернення: 01.05.2026).
21. Pimentel J. F., Murta L., Braganholo V., Freire J. Understanding and improving the quality and reproducibility of Jupyter notebooks // Empirical Software Engineering. 2021. Vol. 26, No. 4. Art. 65. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8106381/> (дата звернення: 02.05.2026).
22. Harris C. R., Millman K. J., van der Walt S. J., et al. Array programming with NumPy // Nature. 2020. Vol. 585. P. 357–362. URL: <https://doi.org/10.1038/s41586-020-2649-2> (дата звернення: 01.05.2026).
23. Lugaresi C., Tang J., Nash H., Hwang J., Yang M. та ін. MediaPipe: A Framework for Building Perception Pipelines. URL: <https://arxiv.org/abs/1906.08172> (дата звернення: 05.05.2026).
24. MediaPipe Hands. URL: <https://chuoling.github.io/mediapipe/solutions/hands.html> (дата звернення: 07.05.2026).

25. Li S., Banerjee S., Banerjee N.K., Dey S. Simultaneous prediction of hand gestures, handedness, and hand keypoints using thermal images, 2023. URL: <https://arxiv.org/abs/2303.01547> (дата звернення: 08.05.2026).

26. Lin J., Zhou A. PyDraw: A GUI Drawing Generator Based on Tkinter and Its Design Concept // arXiv. 2018. URL: <https://arxiv.org/abs/1808.09094> (дата звернення: 07.05.2026).

27. Федорін І. В. Методи та технології обчислювального інтелекту: Практикум [Електронний ресурс] : навч. посіб. для студ. спеціальності 122 «Комп'ютерні науки» / І. В. Федорін; КПІ ім. Ігоря Сікорського. Київ: КПІ ім. Ігоря Сікорського, 2022. 317с. URL :<https://ela.kpi.ua/server/api/core/bitstreams/67822d44-3b72-49c0-95ba-83407c319e65/content> (дата звернення: 11.05.2026).

ДОДАТОК А

Репозиторій проекту

Репозиторій проекту доступний за посиланням <https://shorturl.at/X7kJS>.
Крім того, доступ забезпечено за QR-кодом (рис. А.1).



Рисунок А.1 — QR-код з посиланням на репозиторій